Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Laboratorio de Criptografía Aplicada y Ciberseguridad, CLCERT

Informe Práctica 2

Identificando servidores C2 en la red de dominios chilena con JARM

Equipo: Alejandro Hevia
Eduardo Riveros
Gonzalo Claro

Fecha de entrega: 29 de enero 2021
Santiago, Chile

Índice de Contenidos

1. Abstracto **iii**

2. Introducción **iv**
2.1. Motivación . . . iv
2.2. Marco Teórico . . . iv
2.2.1. Botnet: . . . iv
2.2.2. Protocolo TLS . . . vii
2.2.3. Certificado TLS . . . ix
2.2.4. Red Chilena . . . x
2.2.5. Escaneo Activo . . . x

3. Jarm **xi**

4. Proyecto de Práctica **xiv**
4.0.1. Primer enfoque . . . xiv
4.0.2. Segundo enfoque . . . xvii
4.0.3. Tercer enfoque / Modo de uso . . . xxi

5. Resultados **xxiv**

6. Análisis **xxix**

7. Alcances y Conclusiones **xxxv**

8. Anexos **xxxvi**
8.1. Correos con John Althouse . . . xxxvi
8.2. Enlaces de interés . . . xxxvii
8.3. Códigos . . . xxxviii

Índice de Figuras

1. Modelo cliente-servidor . . . v
2. Nuevo modelo peer to peer . . . v
3. Esquema simple de un ataque DDoS . . . vi
4. Esquema protocolo TLSv1.2 . . . vii
5. Esquema del nuevo protocolo TLSv1.3 . . . viii
6. Niveles de validación: SS, DV y OV/EV . . . ix
7. Escaneo activo (imagen sacada de la tesis de Eduardo Riveros) . . . xi
8. Mensaje *Client Hello* a la izquierda y la respuesta del servidor *Server Hello* a la derecha. [Wireshark] . . . xi
9. fingerprint utilizando Jarm al dominio salesforce.com . . . xii
10. Obtención de los primeros 30 caracteres del fingerprint . . . xii
11. Obtención de los segundos 32 caracteres del fingerprint . . . xiii
12. Ejemplos de fingerprint de Jarm para distintos servidores . . . xiii

13. sitio web de maxmind de donde se pueden obtener las bases de datos de direcciones IP de la red chilena . . . xiv
14. sitio web de Feodo Tracker de donde se pueden obtener las bases de datos de direcciones IP que son potencialmente C2 . . . xv
15. Pequeño extracto de la lista proporcionada por feodo tracker . . . xv
16. Idea de inner join para identificar los servidores C2 de la red chilena . . . xvi
17. Pequeño overview de dominios .CL activos para trabajar . . . xvii
18. Página web de Zones Files . . . xvii
19. Pequeño overview de dominios históricos que fueron C2 para trabajar . . . xviii
20. Esquema del inner join realizado a través del parámetro Jarm . . . xviii
21. Vista previa del github con la lista de fingerprints C2 . . . xix
22. Lista de fingerprints C2 obtenida del github . . . xx
23. El archivo txt a descargar de github como *C2.txt* debe verse similar a la imágen . . . xxi
24. El archivo csv que se ha generado con **raw_txt_to_csv.py**, llamado *Servidores_C2.csv*, debe verse similar a esto . . . xxi
25. Ejemplo de uso . . . xxii
26. Cantidad de fingerprints distintos en los dominios .CL . . . xxiv
27. Frecuencia de algunos fingerprints en los dominios .CL . . . xxiv
28. Idea gráfica de que hay algunos fingerprint muy comunes (outliers) . . . xxv
29. Idea gráfica de que hay algunos fingerprint muy comunes (outliers) . . . xxv
30. Conjunto A: Total dominios .CL con sus fingerprints, Conjunto B: fingerprints de servidores C2 . . . xxvi
31. Conjunto A: Total dominios .CL . . . xxvi
32. Conjunto B: fingerprints de servidores C2 . . . xxvii
33. Dominios .CL que su fingerprint coincide con el de un servidor C2 . . . xxvii
34. Dominios .CL potencialmente C2 y qué tan raros son sus saludos TLS, sus primeros 30 caracteres y sus segundos 32 caracteres . . . xxviii
35. ASN asociado a la IP 143.110.147.106 . . . xxix
36. Más información acerca del ASN asociado a la IP 143.110.147.106 . . . xxix
37. Cantidad de Dominios asociados a la IP 143.110.147.106 . . . xxx
38. ASN asociado a la IP 192.241.132.157 . . . xxx
39. Cantidad de Dominios asociado a 192.241.132.15 . . . xxxi
40. ASN asociado a la IP 198.185.159.144 . . . xxxi
41. Más información acerca del ASN asociado a la IP 198.185.159.144 . . . xxxii
42. Cantidad de Dominios asociados a la IP 198.185.159.144 . . . xxxii
43. ASN asociado a la IP 95.179.243.224 . . . xxxiii
44. Más información acerca del ASN asociado a la IP 95.179.243.224 . . . xxxiii
45. Cantidad de Dominios asociados a la IP 95.179.243.224 . . . xxxiv
46. Primer correo de Jhon Althouse . . . xxxvi
47. Segundo correo de Jhon Althouse . . . xxxvi

Índice de Códigos

1. raw_txt_to_csv.py . . . xxxviii
2. jarm_chile.py . . . xxxix

1. Abstracto

En el campo de la seguridad informática existen infraestructuras llamadas servidores C2 (nombre derivado del inglés Command and Control), estas se encargan de controlar el malware que tienen a su disposición para utilizarlos según sus requerimientos, en particular nos interesa el caso de las botnets. Actualmente no existen métodos universalmente aceptados para identificarlas, si no más bien múltiples alternativas, dentro de las cuales se encuentra Jarm, un fingerprint de protocolos TLS para servidores creado por Salesforce Engineering.

En este trabajo de práctica se hizo uso de la herramienta Jarm, recientemente lanzada a la comunidad de manera libre en github (Noviembre 2020). Esto con el objetivo de intentar generar una lista de potenciales servidores C2, de un conjunto de dominios considerados de la red chilena extraídos desde los registros desde NIC Chile. Por otro lado se espera aprender más acerca de los alcances de Jarm y sus usos.

Los resultados obtenidos al día 29/01 arrojan 6 dominios .cl, pero dadas las limitaciones de Jarm, no se puede llegar a una conclusión inmediata acerca de si son realmente servidores C2 o no. Es por esto que se propone solamente dejar la lista como una de potenciales servidores C2 y no como algo absoluto. Por otro lado se propone hacer una inspección manual de estos dominios y los servidores que los alojan, para así poder tener una mejor idea acerca de su peligrosidad.

2. Introducción

2.1. Motivación

Las botnets es una de las amenazas más comunes de las que podemos ser víctimas y también una de las más peligrosas. Mediante una botnet se pueden lograr diversos objetivos de manera remota por parte de un hacker, como lo puede ser un ataque DDoS, engañar redes de publicidad, enviar spam a correos electrónicos o minar criptomonedas. Lamentablemente todas estas prácticas son cada día más comunes, mientras que el identificar los servidores que controlan estas botnets (servidores C2) es cada vez más difícil.

Es por este motivo, del auge de las botnets y la dificultad para identificar los servidores que las controlan, que la necesidad de encontrar más y mejores alternativas para identificar servidores C2 se hace algo de suma importancia. Es en este contexto que Jarm surge como una nueva alternativa de identificar servidores C2 en base a comparar fingerprints de los protocolos TLS, que hasta ahora no son algo que se suela modificar mucho, por lo que abre una nueva posibilidad (probablemente momentánea) de identificar algunos servidores potencialmente malignos en la red Chilena.

2.2. Marco Teórico

A continuación se explican algunos conceptos clave para el entendimiento del trabajo:

2.2.1. Botnet:

El nombre botnet surge de la combinación de los términos bot y network, ya que es una red de ordenadores infectados que pueden trabajar en conjunto para lograr un objetivo en particular. Estas son controladas y manejadas comúnmente desde un servidor de comando y control (en general llamado por su nombre en inglés command and control o por su abreviación C2) el cual actúa como centro de comando para las mismas, indicando el objetivo a realizar.

El peligro de una botnet es proporcional a la cantidad de equipos infectados que posee bajo su control, conocidos como equipos zombie, los cuales realizaran la tarea que el hacker tenga planeada. Es así como una botnet consiste en dos partes: la red de ordenadores zombies y el ordenador maestro que las controla (servidor C2). Por otro lado uno puede ser afectado de dos formas por una botnet: siendo víctima de un ataque perpetrado por botnets o bien formando parte de una sin saberlo.

Una de las formas de manejar las botnets, aunque ya cada vez en menor uso, es mediante un modelo cliente-servidor, en el cual los ordenadores infectados eran controlados por un único servidor (o sitio web). Esta manera es la más sencilla de contraarrestar, ya que supone identificar y eliminar solamente un servidor maligno para hacer caer el sistema.

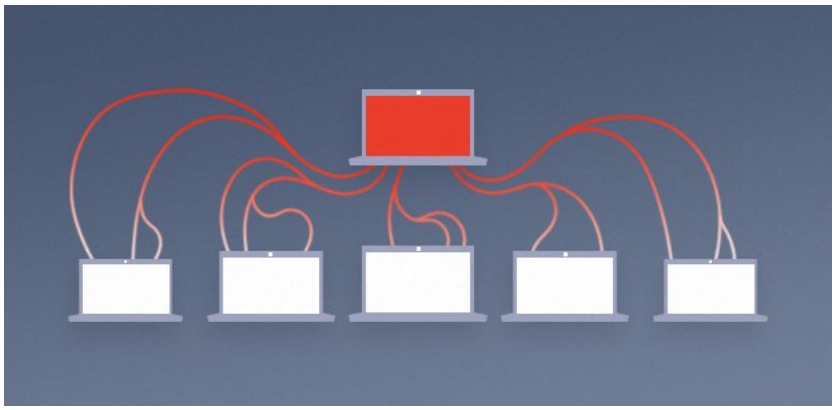

Figura 1: Modelo cliente-servidor

Actualmente se suele utilizar un modelo peer to peer para poner solución al "problema"del modelo cliente-servidor. En este sistema cada ordenador infectado se comunica directamente a otros de la red de la botnet de manera que todo el sistema está unido. Así los ordenadores infectados actúan tanto como cliente como servidor, enviando y recibiendo comandos maliciosos por igual y evitando tener un punto único de falla.

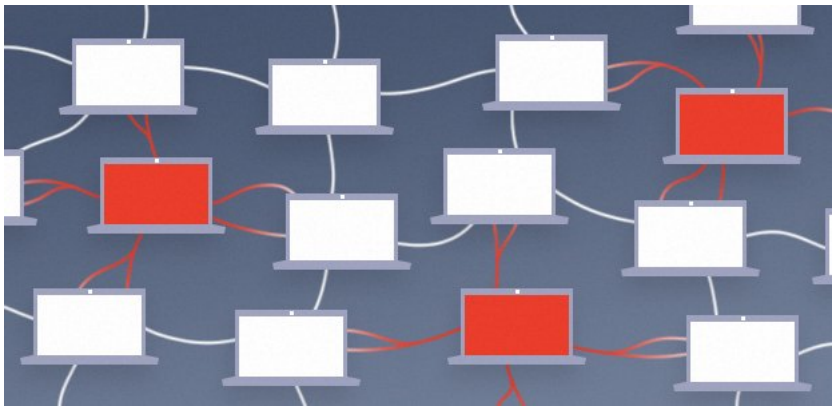

Figura 2: Nuevo modelo peer to peer

Una forma común de verse infectado por una botnet es mediante un troyano, el cual se puede obtener mediante phishing en un correo electrónico, software pirateado, anuncios maliciosos (adware), etc. Este troyano lo que hará es crear una *puerta falsa* o *backdoor*, para que así el hacker pueda acceder a ciertos elementos del ordenador y controlarlos cuando él estime conveniente, permaneciendo hasta entonces inactivo para pasar desapercibido. El troyano de por si no busca expandirse por la red, pero hay ciertos tipos de malware que si lo pueden hacer, los denominados worms (ver el caso de la botnet Storm). Sea cual sea el método, una vez el hacker pueda infectar un número de ordenadores considerable, es que tendrá disponible la botnet para su uso.

Como mencionábamos anteriormente, con una botnet se pueden ejecutar diversos ataques:

- **Difundir spam por correo electrónico:** Millones de correos son enviados día a día por botnets para ataques de phishing, en donde estos correos pueden servir para expandir la

botnet o bien para enviar archivos de malware como podría ser ransomware entre muchos otros.

- **Difundir malware rápidamente:** Una botnet permite a los virus llegar a un gran número de ordenadores rápidamente, ya sea mediante spam en correo electrónico o por medio de una red abierta.

- **Ataques distribuidos de denegación de servicio o DDoS:** Son ataques que intentan impedir a los usuarios el acceso a un servicio (ej: sitio web) mediante la satuación de la red o el servidor con múltiples sistemas (gran escala). Algunas formas de ataques DDoS son los *ping flood* en donde se envían toneladas de paquetes ping a un sistema, en particular se envían paquetes ICM echo request, ya que ping espera un número de respuestas. Si un servidor o sitio web no puede seguirle el ritmo es propenso a verse saturado y quedar fuera de funcionamiento. Similar es la *SYN flood*, donde recordamos que para hacer una conexión TCP, un cliente envía un paquete SYN a un servidor al que quiere conectarse, a continuación el servidor envía un mensaje SYN-ACK, para posteriormente el cliente responder con un mensaje ACK. En una SYN flood, al servidor se le bombardea con paquetes SYN, a lo que este responde con los paquetes SYN-ACK, pero el atacante no responderá con mensajes ACK. Esto quiere decir que la conexión permanece abierta y acapara recursos del servidor, por lo que otros usuarios no podrán conectarse. Este tipo de ataques SYN también son llamados *half-open attacks*.

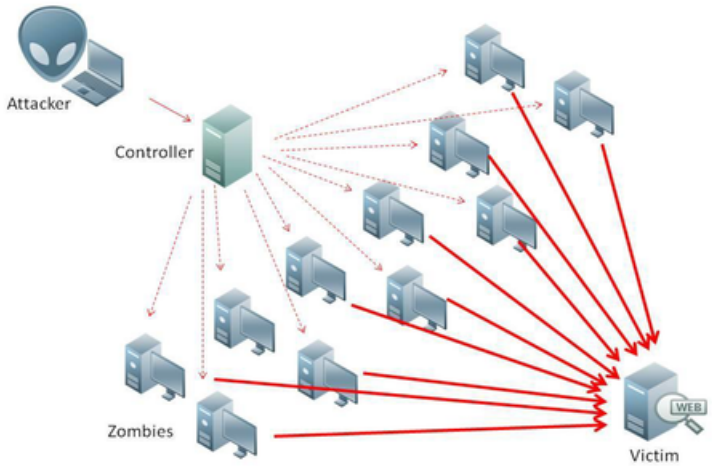

Figura 3: Esquema simple de un ataque DDoS

- **Averiguar contraseñas:** Un problema que tiene intentar averiguar una contraseña mediante fuerza bruta o ataques de diccionario es la cantidad de intentos con los que se cuenta antes de que se bloquee la cuenta asignados a una cierta dirección IP. Es por esto que contar con una gran cantidad de ordenadores con IPs distintas puede ayudar a descifrar casi cualquier contraseña (teniendo la cantidad de ordenadores y tiempo suficientes). Lo bueno es que contraseñas seguras tardarían millones de años en ser descifradas, por lo que es importante concientizar a los usuarios acerca de la importancia de las mismas.

- **Minar criptomonedas:** Uno de los usos más comunes de las botnets es para minar criptomonedas, que si bien no es un ataque directamente, si que ralentizará el equipo afectado ya que se

consumen recursos para realizar esa labor. Esto es comúnmente conocido como *cryptojacking* y es una tendencia al alza dado lo difícil que es detectarla y su inofensiva apariencia.

2.2.2. Protocolo TLS

El protocolo TLS es un protocolo criptográfico cuyo objetivo principal es ”proporcionar privacidad e integridad de datos entre dos aplicaciones que se comunican”. La primera versión de TLS se lanzó en enero de 1999 para reemplazar el ahora obsoleto protocolo SSL y agosto de 2008 se lanzó la versión más extendida de TLS que es TLSv1.2.

El protocolo TLS se encuentra debajo de la capa de aplicación y encima de la capa de transporte, principalmente TCP. El uso principal de TLS hoy en día es cifrar el tráfico HTTP con el que forma el protocolo HTTPS; sin embargo, el uso de TLS no se limita solo a HTTP: en teoría, cualquier protocolo de capa de aplicación puede hacer uso de TLS.

El protocolo TLSv1.2 funciona de la siguiente manera:

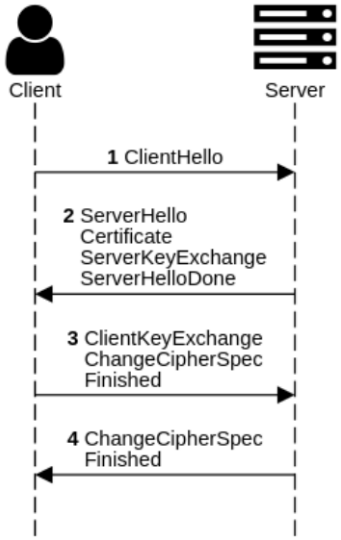

Figura 4: Esquema protocolo TLSv1.2

- El cliente envía un **ClientHello message** que contiene: una lista de conjuntos de cifrado adimitidos por el cliente, una lista de métodos de compresión y una lista de extensiones que utiliza el cliente para comunicar información adicional al servidor (como nombre del host de destino, etc).
- El servidor responde con varios mensajes: (a) **ServerHello**: contiene el conjunto de cifrado y el método de compresión seleccionado por el servidor. (b) **Certificate**: contiene una cadena de certificados TLS que prueban la propiedad de una clave pública. (c) **ServerKeyExchange**: contiene información que permite al cliente comunicar un secreto de premaster (opcional, útil cuando el servidor no tiene certificado o si su clave pública es solo para firmar mensajes). (d) **ServerHelloDone**: informa al cliente que el servidor ha terminado de enviar mensajes en esta primera parte del *handshacke*.

- El cliente responde con: (a) **ClientKeyExchange**: que contiene información que permite al servidor calcular la clave de sesión simétrica final (una clave de sesión encriptada con la clave pública del servidor en la mayoría de los casos) (b) **ChangeCipherSpec**: informa al servidor que todos los mensajes posteriores serán encriptados con la clave de sesión (c) **Finished**: informa al servidor que el protocolo de enlace TLS fue exitoso para el cliente.
- Finalmente, el servidor finaliza la negociación con: (a) **ChangeCipherSpec**: informa al cliente que todos los mensajes posteriores serán encriptados con la clave de sesión. (b) **Finished**: informa al cliente que el protocolo de enlace TLS fue exitoso para el servidor.

En 2018 fue lanzado el protocolo TLSv1.3 donde los mayores cambios son:

- Eliminar los cifrados y algoritmos hash obsoletos: SHA1, MD5, DES, etc.
- Reducir el número de mensajes necesarios para el protocolo de enlace.
- Todos los mensajes posteriores al **ServerHello** ahora están cifrados.

Así el protocolo TLSv1.3 requiere sólo 3 mensajes:

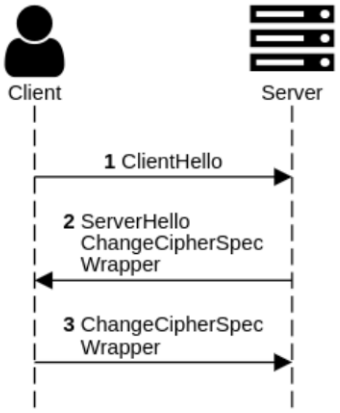

Figura 5: Esquema del nuevo protocolo TLSv1.3

- El mensaje **ClientHello** es idéntico al de TLSv1.2 pero también lleva una lista de claves públicas.
- El servidor responde con múltiples mensajes: (a) **ServerHello** contiene la misma información que en TLSv1.2, así como una clave pública. A partir de la clave pública del cliente y la clave pública del servidor, se deriva una clave compartida y se utiliza solo para cifrar el resto del protocolo de enlace. (b) **ChangeCipherSpec** tiene el mismo propósito que en TLSv1.2, es importante señalar que los mensajes a continuación están cifrados, incluido el certificado del servidor. (c) **Wrapper**: abarca los mensajes del servidor también presentes en TLSv1.2: *Finished*, *Certificate*, etc.
- El cliente finaliza el *handshake* con: (a) **ChangeCipherSpec**: que tiene el mismo propósito de TLSv1.2 y posterior a este mensaje todo va cifrado (b) **Wrapper**: abarca los mensajes del servidor también presentes en TLSv1.2: *Finished*, mensajes opcionales, etc.

2.2.3. Certificado TLS

Los certificados de servidor son documentos electrónicos que vinculan una clave pública a un servidor, firmados digitalmente por una entidad llamada *Certificate Authority*. Los clientes que reciben un certificado de servidor durante un protocolo de enlace TLS deben verificar que sea válido y que haya sido firmado por una autoridad de confianza. Si el certificado ha sido firmado por una organización desconocida, el cliente tiene que ascender en la cadena de certificados hasta que se encuentre uno de una autoridad de confianza, que por construcción valida todos los certificados debajo de él en la cadena. Cuando finalmente se ha verificado la legitimidad del servidor, la clave pública del servidor se puede utilizar para cifrar y compartir una clave de sesión simétrica.

Un certificado TLS generalmente sigue el estándar X.509, que contiene múltiples campos, entre los que destacan:

- **Issuer**: entidad que verificó la legitimidad del servidor y emitió el certificado (una *Certificate Authority* en la mayoría de los casos).
- **Validity**: contiene la fecha desde que el certificado es válido hasta la fecha en que caduca.
- **Subject**: el beneficiario del certificado.
- **Subject Public Key Info**: contiene dos subcampos que indican el algoritmo de la clave pública del servidor y la clave pública en sí.
- **Extensions** (opcional): contiene varios campos que indican cómo se debe usar el certificado e información adicional sobre el certificado.
- **Certificate Signature Algorithm** y **Certificate Signature Value**: el algoritmo de firma y la firma del emisor del organismo del certificado
- **Fingerprint** (no es una parte real del certificado): el hash de todo el certificado (generalmente SHA256 o SHA1).

Sin embargo, los certificados no tienen el mismo nivel de validación y los navegadores muestran diferentes símbolos para cada uno de ellos: los candados de Chrome se muestran en la siguiente figura:

Figura 6: Niveles de validación: SS, DV y OV/EV

Los niveles de validación se pueden agrupar en cuatro categorías:

- **Self-Signed (SS)**: Este es el nivel más bajo de validación, ya que significa que el propio destinatario del certificado ha realizado una copia de seguridad del certificado. Por lo general, las autoridades de certificación utilizan certificados autofirmados para compartir su clave pública, y los navegadores están configurados para confiar en estos certificados en particular (llamados certificados raíz). De entidades distintas de las CAs, dichos certificados no proporcionan ninguna prueba de la propiedad de la clave y los navegadores suelen mostrar advertencias sobre ellos.
- **Domain Validation (DV)**: La CA solo ha verificado la conexión entre una clave pública y un nombre de dominio sin más controles de identidad.
- **Organization Validation (OV)**: La CA ha verificado el nombre de dominio y alguna información sobre la organización antes de emitir el certificado.
- **Extended Validation (EV)**: El certificado "más fuerte", donde la CA ha realizado una investigación exhaustiva sobre la organización, además de verificar el nombre de dominio.

2.2.4. Red Chilena

En la red de direcciones IPv4 existen a lo más 2^{32} dispositivos conectados a internet que pueden prestar servicios (límite dado por la asignación de estas direcciones). Es por esto que se quiere trabajar sobre un subconjunto de internet que llamaremos red chilena.

En este trabajo entenderemos por la Red Chilena al conjunto de dominios .cl suministrados por NIC Chile, centro perteneciente a la Facultad de Ciencias Físicas y Matemáticas de la Universidad de Chile y encargado de la administración del Registro de Nombres de Dominio .CL, que identifica a Chile en la red Internet.

En particular se hizo uso de un registro de 626.681 dominios .CL entregado por el CLCERT.

2.2.5. Escaneo Activo

Por escaneo se entiende generalmente el analizar, mediante algún programa, los puertos que una máquina conectada a la red puede tener tanto abiertos, como cerrados o protegidos. Se utiliza para detectar qué servicios comunes está ofreciendo la máquina y posibles vulnerabilidades de seguridad según los puertos que esta tenga abiertos. También se pueden llegar a detectar cosas como el sistema operativo que está ejecutando la máquina entre otras cosas. Es usado por administradores de sistemas para analizar posibles problemas de seguridad, pero también es utilizado por usuarios malintencionados que intentan comprometer la seguridad de la máquina o la red.

Un escaneo activo sobre la red considera el envío de mensajes a los dispositivos que se desean estudiar, por lo que este tipo de escaneo genera tráfico sobre la red proporcional a la cantidad de equipos a escanear. A su vez, cada máquina escaneada consume recursos para responder (o ignorar) las consultas del escaneo, por lo que resulta en un estudio algo invasivo del que hay que hacer buen uso.

Figura 7: Escaneo activo (imagen sacada de la tesis de Eduardo Riveros)

Es por esto que se recomienda no realizar labores de escaneo activo (como lo son los fingerprint de Jarm) con regularidad, o si este es el caso, avisar de su realización a las entidades correspondientes.

3. Jarm

Jarm es una herramienta de escaneo activa que genera fingerprints del protocolo TLS (Transport Layer Security) de los servidores escaneados. Ante un mensaje inicial TLS Client Hello del cliente, cada servidor responderá con un mensaje TLS Server Hello de una forma única dada su configuración de puertos, sistema operativo y su versión, librerías utilizadas, etc.

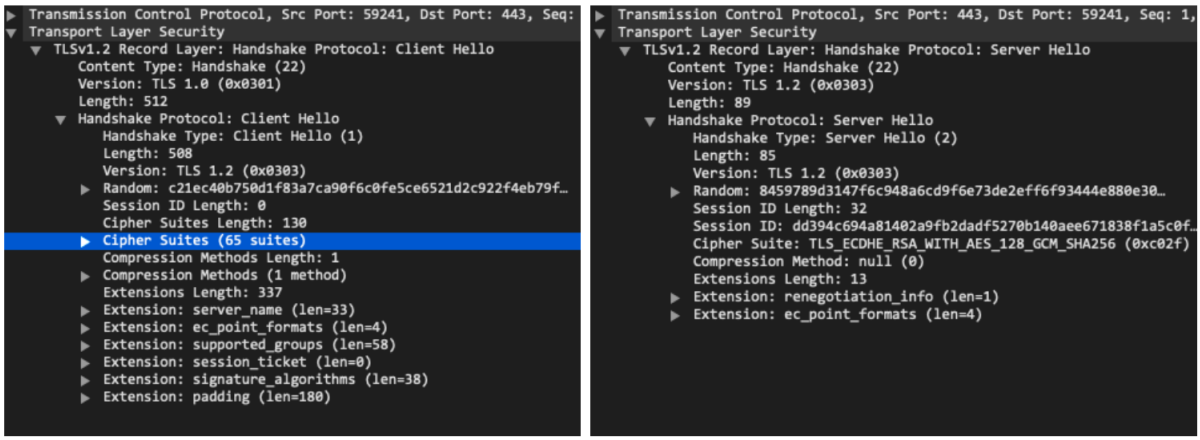

Figura 8: Mensaje *Client Hello* a la izquierda y la respuesta del servidor *Server Hello* a la derecha. [Wireshark]

Jarm funciona enviando activamente 10 paquetes TLS Client Hello a un servidor TLS de destino y capturando atributos específicos de las respuestas TLS Server Hello. Los 10 paquetes TLS Client Hello están diseñados especialmente para enviar diferentes versiones, cifrados y extensiones de TLS en diferentes órdenes para así obtener respuestas únicas en los servidores TLS (ej: ver compatibilidad con TLSv1.3, ver si el TLSv1.3 responderá a un llamado TLSv1.2, ver qué cifrados escoge el servidor, si los más fuertes o débiles, etc). Finalmente las 10 respuestas luego se procesan para producir el fingerprint de JARM.

El fingerprint de Jarm consta de 62 caracteres, como se muestra en la siguiente imagen:

```
test@kobayashi-maru:~/jarm$ python3 jarm.py salesforce.com
Domain: salesforce.com
Resolved IP: 23.1.99.130
JARM: 2ad2ad0002ad2ad00042d42d000000c9f11021662addffd4606e9f59a1ec98
```

Figura 9: fingerprint utilizando Jarm al dominio salesforce.com

Los primeros 30 caracteres están formados por el cifrado y la versión TLS elegidos por el servidor para cada uno de los 10 saludos de cliente enviados. Un ”000” indica que el servidor se negó a negociar con el saludo del cliente. A continuación se muestran las partes que generan estos primeros 30 caracteres del fingerprint a partir de los 10 saludos TLS de Jarm:

```
test@kobayashi-maru:~/jarm$ python3 jarm.py -v salesforce.com
Domain: salesforce.com
Resolved IP: 104.109.10.129
JARM: 2ad2ad0002ad2ad00042d42d000000c9f11021662addffd4606e9f59a1ec98
Scan 1: c030|0303|http/1.1|ff01-0000-0001-000b-0023-0010,
Scan 2: c030|0303|http/1.1|ff01-0000-0001-000b-0023-0010,
Scan 3: |||,
Scan 4: c030|0303|http/1.0|ff01-0000-0001-000b-0023-0010,
Scan 5: c030|0303|http/1.0|ff01-0000-0001-000b-0023-0010,
Scan 6: |||,
Scan 7: 1302|0303||002b-0033,
Scan 8: 1302|0303||002b-0033,
Scan 9: |||,
Scan 10: |||
```

Figura 10: Obtención de los primeros 30 caracteres del fingerprint

Los 32 caracteres restantes son un hash SHA256 truncado de las extensiones acumulativas enviadas por el servidor, ignorando los datos del certificado x509.

```
test@kobayashi-maru:~/jarm$ python3 jarm.py -v salesforce.com
Domain: salesforce.com
Resolved IP: 104.109.10.129
JARM: 2ad2ad0002ad2ad00042d42d000000c9f11021662addffd4606e9f59a1ec98
Scan 1: c030|0303|http/1.1|ff01-0000-0001-000b-0023-0010,
Scan 2: c030|0303|http/1.1|ff01-0000-0001-000b-0023-0010,
Scan 3: |||,
Scan 4: c030|0303|http/1.0|ff01-0000-0001-000b-0023-0010,
Scan 5: c030|0303|http/1.0|ff01-0000-0001-000b-0023-0010,
Scan 6: |||,
Scan 7: 1302|0303||002b-0033,
Scan 8: 1302|0303||002b-0033,
Scan 9: |||,
Scan 10: |||
```

Figura 11: Obtención de los segundos 32 caracteres del fingerprint

Al comparar fingerprints de JARM, si los primeros 30 caracteres son iguales pero los últimos 32 son diferentes, esto significaría que los servidores tienen configuraciones muy similares, aceptando las mismas versiones y cifrados, aunque no exactamente iguales dado que las extensiones son diferentes. Mientras que si los 62 caracteres del fingerprint son idénticos, significa que son muy similares los servidores y funcionan de maneras muy similares. Un ejemplo es que los fingerprints de google.com y youtube.com son iguales entre si, pero diferentes a los de facebook.com e instagram.com (iguales entre si).

Domain	JARM Fingerprint
google.com	27d40d40d29d40d1dc42d43d00041d4689ee210389f4f6b4b5b1b93f92252d
youtube.com	27d40d40d29d40d1dc42d43d00041d4689ee210389f4f6b4b5b1b93f92252d
blogger.com	27d40d40d29d40d1dc42d43d00041d4689ee210389f4f6b4b5b1b93f92252d
facebook.com	27d27d27d29d27d1dc41d43d00041d741011a7be03d7498e0df05581db08a9
instagram.com	27d27d27d29d27d1dc41d43d00041d741011a7be03d7498e0df05581db08a9
oculus.com	29d29d20d29d29d21c41d43d00041d741011a7be03d7498e0df05581db08a9

Cipher picked and TLS version, SHA256 of TLS extensions

Figura 12: Ejemplos de fingerprint de Jarm para distintos servidores

4. Proyecto de Práctica

Como vimos en la sección anterior, servidores con protocolos TLS iguales, tienen fingerprints iguales. Es por esto que Jarm nos puede servir de herramienta para clasificar y agrupar servidores que tienen estos protocolos TLS en común, en particular nos interesará ver los servidores de la red chilena que posean un fingerprint igual a los fingerprints de servidores command and control (C2). Esto para generar una lista de potenciales dominios C2 dentro del subconjunto de dominios .CL.

De esta forma el proyecto de práctica fue utilizar Jarm para obtener una lista de dominios C2 en la red chilena.

4.0.1. Primer enfoque

Un primer enfoque fue extraer las direccioes IP de la red chilena desde la plataforma **MaxMind Geolite 2**, para luego hacer un primer filtro sobre estas direcciones utilizando **Zmap** para escanear la red en busca de, por ejemplo, servidores con el puerto 443 abierto de https.

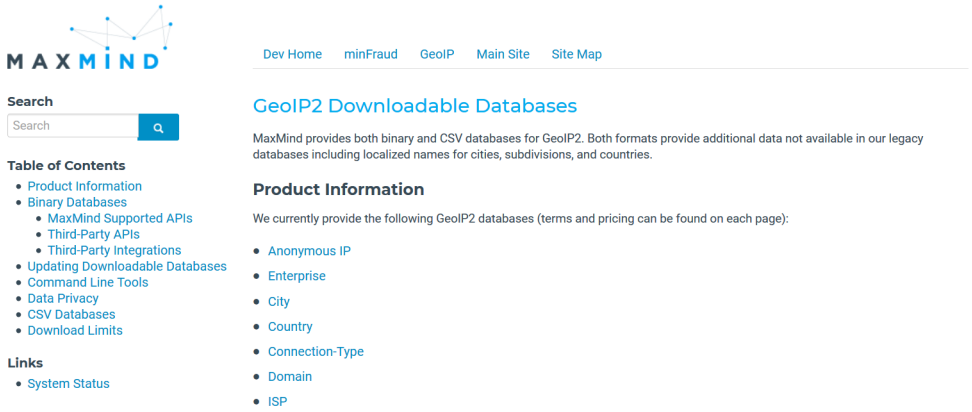

Figura 13: sitio web de maxmind de donde se pueden obtener las bases de datos de direcciones IP de la red chilena

Luego de este primer filtro con Zmap se podría utilizar Jarm sobre las direcciones IP obtenidas, para así tener un fingerprint de cada una para posteriormente comparar con los fingerprints de una lista de bloqueo de IPs obtenida del sitio **Feodo Tracker**.

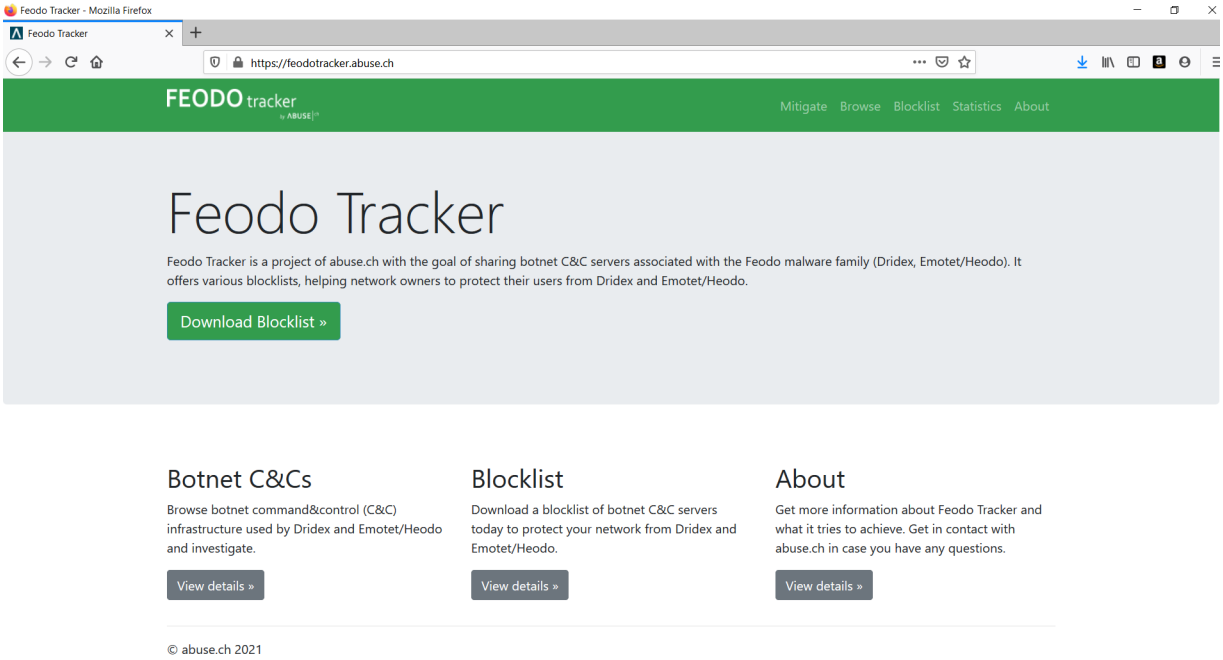

Figura 14: sitio web de Feodo Tracker de donde se pueden obtener las bases de datos de direcciones IP que son potencialmente C2

Feodo Tracker proporciona una lista actualizada de IPs potencialmente maliciosas cada 5 minutos, por lo que es una fuente en constante actualización.

```
##################################################################
# abuse.ch Feodo Tracker Botnet C2 IP Blocklist (recommended)   #
# Last updated: 2021-01-28 08:17:12 UTC                         #
#                                                               #
# Terms Of Use: https://feodotracker.abuse.ch/blocklist/        #
# For questions please contact feodotracker [at] abuse.ch       #
##################################################################
#
# DstIP
67.213.75.205
192.73.238.101
51.178.161.32
194.58.98.196
142.4.6.57
93.186.200.154
162.144.127.197
5.9.178.143
37.139.2.140
49.212.179.180
23.160.192.125
221.126.244.72
157.7.166.26
212.129.24.83
208.71.173.207
77.220.64.37
69.64.62.4
162.241.44.26
111.230.104.169
217.79.184.243
194.150.118.7
199.66.90.63
62.75.168.106
82.165.152.127
```

Figura 15: Pequeño extracto de la lista proporcionada por feodo tracker

De esta manera una vez obtenidos los fingerprints de la red chilena y los fingerprints de la lista de bloqueo, bastaría con hacer un inner join entre ambos conjuntos mediante el campo del fingerprint de Jarm.

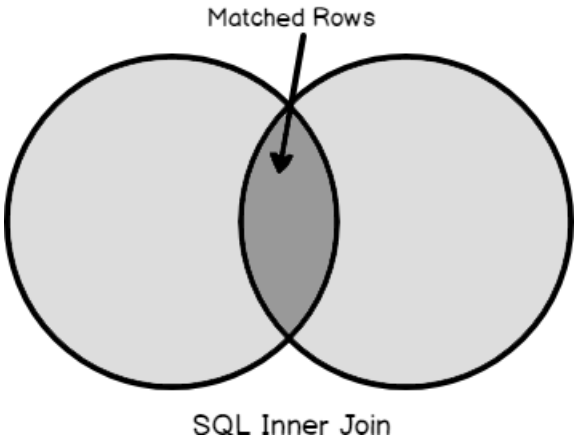

Figura 16: Idea de inner join para identificar los servidores C2 de la red chilena

El problema de este enfoque (y me di cuenta experimentalmente) es que una dirección IP puede albergar más de un dominio, por lo que el fingerprint sobre direcciones IP se vuelve arbitrario ya que depende exclusivamente de a cual dominio termine redireccionando. Así una IP definida como maliciosa puede redireccionar a un dominio limpio y entregar un fingerprint no sospechoso, o bien, una IP que no se encuentre en la lista (definida como limpia) podría también redirigir a un dominio malicioso y entregar un fingerprint sospechoso.

Dado este problema de que a una dirección IP pueden asignarse múltiples dominios, es que se abandonó la idea de intentar hacer un inner join con dominios y se procedió a utilizar directamente dominios .CL y dominios maliciosos de botnets.

4.0.2. Segundo enfoque

Ahora en vez de utilizar las direcciones IP de maxmind, se trabajó con los dominios .CL activos otorgados por NIC Chile. Un total de 626.681 dominios .CL, de los cuales al filtrar los que no hacen conexión con Jarm (fingerprint 000..) nos da un total de **292.439** dominios activos.

	Dominio	IP	Jarm
1	007gps.cl	190.4.193.170	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...
4	0101.cl	190.107.177.235	25d3fd00025d25d00025d3fd25d25d7e5e84a97e625d04...
5	0000.cl	168.195.128.167	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...
7	009.cl	200.35.157.92	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...
14	06publicidad.cl	201.148.107.13	2ad2ad0002ad2ad22c2ad2ad2ad2ad6a7bd8f51d54bfc0...
...	...	...	...
626588	zuletaygarciacontadores.cl	131.72.236.33	07d14d16d21d21d07c42d43d000000eeb722fe81b496eb...
626597	zuntao.cl	131.72.236.178	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...
626604	zuritapublicidad.cl	131.72.236.63	15d3fd16d29d29d00042d43d0000001ae0802418786940...
626606	zuvar.cl	131.72.236.168	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...
626658	zonanotebook.cl	131.72.236.58	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...

292439 rows × 3 columns

Figura 17: Pequeño overview de dominios .CL activos para trabajar

Por otro lado, los dominios asociados a botnets para poder realizar el inner join inicialmente pensado, se sacaron de la página **Zones Files**.

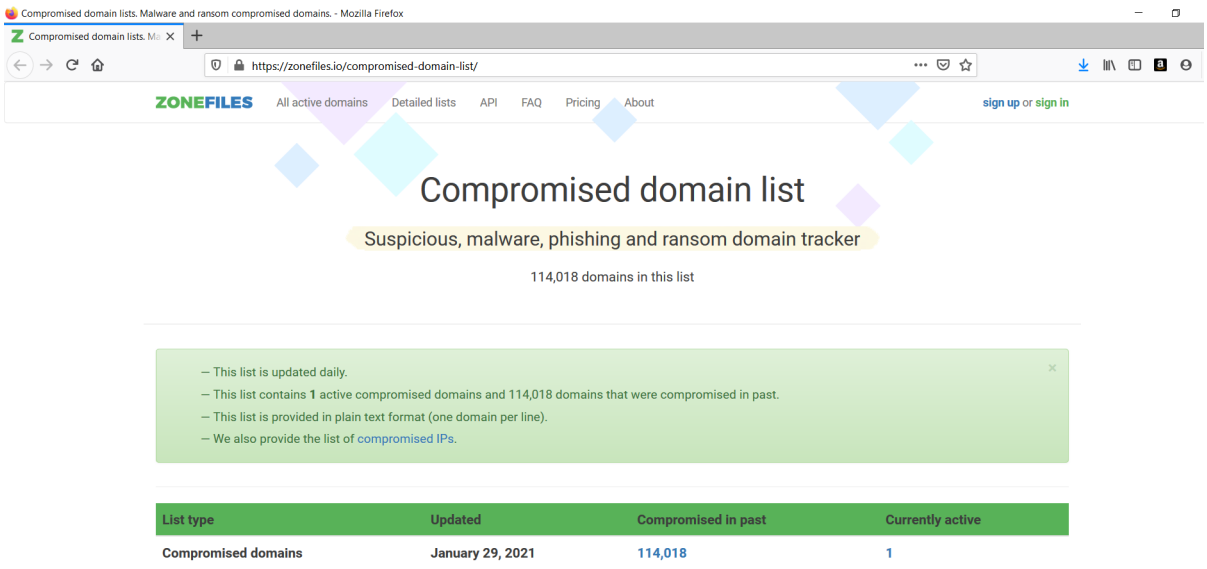

Figura 18: Página web de Zones Files

El archivo descargable desde zones files es un archivo txt como el que se muestra a continuación, son 114.018 dominios sospechosos históricos totales.

```
# Compromised domains / historical list (domains compromised in past). Downloaded from Zonefiles.io
afobal.cl
alvoportas.com.br
atmape.ru
bestdove.in.ua
blogerjijer.pw
bright.su
citricbenz.website
danislenefc.info
dau43vt5wtrd.tk
dzitech.net
ebesee.com
flex.comonwealthplc.com
gmailsecurityteam.com
goodbytoname.com
gzhueyuatex.com
hruner.com
hui-ain-apparel.tk
ice.ip64.net
interlogistics.com.vn
ivansaru.418.com1.ru
jad.fisbonline.com
jangasm.org
jump1ng.net
kesikelyaf.com
kntksales.tk
liuz112.ddns.net
luenhinpearl.com
machine.cu.ma
maminoleinc.tk
metalexvietnamreed.tk
nasscomminc.tk
```

Figura 19: Pequeño overview de dominios históricos que fueron C2 para trabajar

Al filtrar los dominios de la lista de servidores C2 que no realizan conexión quedan un total de **1.805** dominios sospechosos históricos activos. Si realizamos el inner join entre ambos conjuntos obtenemos 143.743 dominios .CL que tienen un fingerprint igual al de uno de los dominios C2.

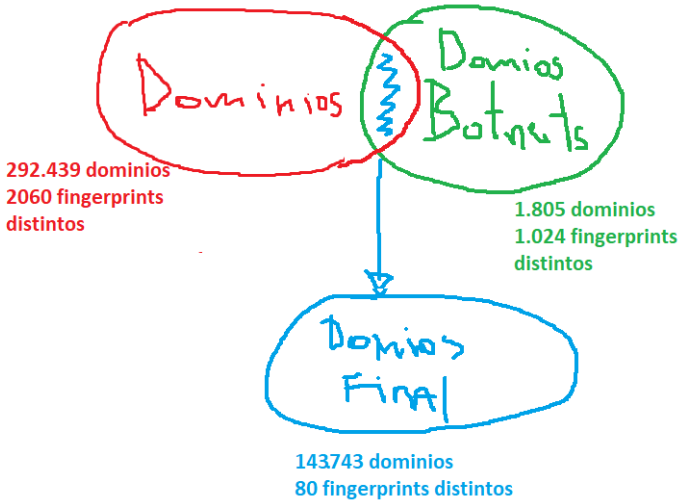

Figura 20: Esquema del inner join realizado a través del parámetro Jarm

La elevada cantidad de resultados obtenidos se debe a un problema del archivo entregado por Zones Files. Este archivo no es uno actualizado si no más bien un registro histórico, por lo que muchos de esos dominios pueden ser (o no) dominios maliciosos actualmente. Es dada esta ambiguedad que se presentan una gran cantidad de falsos positivos, porque basta un dominio de la lista que ya no sea servidor C2 y que posea un fingerprint común, para que se generen una gran cantidad de falsos positivos al hacer un inner join con los dominios .CL.

Se envió un correo a Jhon Althouse (creador de Jarm) ya que en el post de Jarm mencionan que ellos realizan el match de los fingerprints de los dominios, con un set de fingerprints conocidos de servidores C2. En el correo se le pregunta acerca de cómo los obtuvieron y maneras de utilizar Jarm correctamente.

Básicamente su respuesta (correo 2 del anexo) fue una lista de fingerprints de Jarm de servidores C2 que ellos tenían y que iban a actualizar supuestamente. Algunos de estos fingerprints se chequearon mediante las implementaciones SSL de los servidores y hay algunos trabajos como el que menciona de Cobalt Strike de inspecciones más rigurosas. Además proporciona el github donde se encuentran algunos de los fingerprints de servidores C2 que recomienda para trabajar https://github.com/cedowens/C2-JARM.

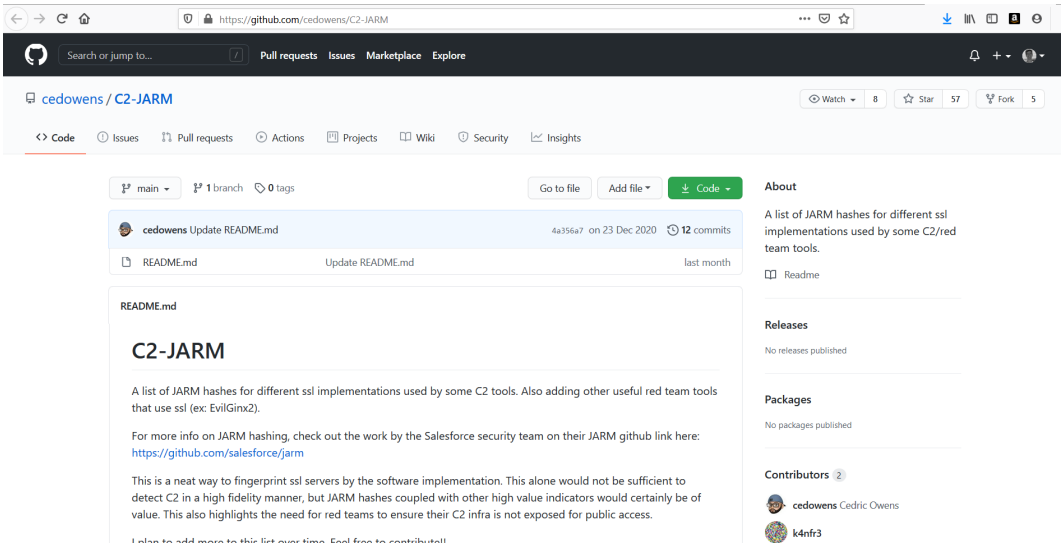

Figura 21: Vista previa del github con la lista de fingerprints C2

La lista completa, actualizada por última vez el 23 de Diciembre, se muestra a continuación:

C2/RED TEAM TOOL	SSL IMPLEMENTATION TESTED	JARM HASH	LINK TO TOOL
Mythic	python 3 w/aiohttp 3	2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb	https://github.com/its-a-feature/Mythic
Metasploit ssl listener	ruby 2.7.0p0	07d14d16d21d21d00042d43d000000aa99ce74e2c6d013c745aa52b5cc042d	https://github.com/rapid7/metasploit-framework
Metasploit ssl listener	ruby	07d14d16d21d21d07c42d43d000000f50d155305214cf247147c43c0f1a823	https://github.com/rapid7/metasploit-framework
Cobalt Strike	Java 11	07d14d16d21d21d07c42d41d00041d24a458a375eef0c576d23a7bab9a9fb1	https://www.cobaltstrike.com/
Merlin	go 1.15.2 linux/amd64	29d21b20d29d29d21c41d21b21b41d494e0df9532e75299f15ba73156cee38	https://github.com/Ne0nd0g/merlin
Deimos	go 1.15.2 linux/amd64 with github.com/gorilla/websocket package	00000000000000000041d00000041d9535d5979f591ae8e547c5e5743e5b64	https://github.com/DeimosC2/DeimosC2
MacC2	python 3.8.6 w/aiohttp 3	2ad2ad0002ad2ad22c42d42d000000faabb8fd156aa8b4d8a37853e1063261	https://github.com/cedowens/MacC2
MacC2	python 3.8.2 w/aiohttp 3	2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb	https://github.com/cedowens/MacC2
MacShellSwift	python 3.8.6 socket	2ad00000000000000000000000000eeebf944d0b023a00f510f06a29b4f46	https://github.com/cedowens/MacShellSwift
MacShell	python 3.8.6 socket	2ad00000000000000000000000000eeebf944d0b023a00f510f06a29b4f46	https://github.com/cedowens/MacShellSwift
Sliver	go 1.15.2 linux/amd64	2ad2ad0002ad2ad00041d2ad2ad41da5207249a18099be84ef3c8811adc883	https://github.com/BishopFox/sliver
EvilGinx2	go 1.10.4 linux/amd64	20d14d20d21d20d20c20d14d20d20daddf8a68a1444c74b6dbe09910a511e6	https://github.com/kgretzky/evilginx2
Shad0w	python 3.8 flask	2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb	https://github.com/bats3c/shad0w
Get2	N/A	07d19d12d21d21d07c07d19d07d21da5a8ab90bcc6bf8bbc6fbec4bcaa8219	

Figura 22: Lista de fingerprints C2 obtenida del github

Lo que fue decidor para dejar de lado este segundo enfoque, es que ningún dominio de la lista de Zones Files tiene el mismo fingerprint que estos entregados por Salesforce y que sabemos que estos últimos están actualizados en comparación a los obtenidos de Zones Files que son históricos. Por otro lado, en distintos sitios he visto (en particular uno de IBM que se adjunta en el anexo) que la dificultad de generar listas de dominios de servidores C2 es muy alta, ya que los dominios son tan baratos en empresas de hosting que los hackers tienen muchos dominios para utilizar como comando y control, los cuales cambian muchas veces en el día de uno a otro de forma dinámica para que no sean fácilmente rastreables o añadirlos a una lista de bloqueos de dominios sea poco fructífero.

Es por eso que el enfoque de utilizar directamente los fingerprints de los servidores C2 se hace buena idea, ya que corresponden directamente a la configuración TLS de esos servidores, lo que no es tan fácil de cambiar como el dominio en donde se alojan. Es por esto que se propone un tercer enfoque al problema.

4.0.3. Tercer enfoque / Modo de uso

Este enfoque consiste en realizar el mismo inner join pero ahora entre el total de dominios .CL activos y la lista de fingerprints de servidores C2 proporcionada por Salesfoce Engineering (figura 22).

Se recomienda extraer la lista actualizada descargando el archivo raw del github mencionado anteriormente (enlace 9 del anexo) como un archivo txt y llamarlo *C2.txt*. Luego utilizar el archivo **raw_txt_to_csv.py** para su paso de txt a un csv ordenado y limpio. Este archivo csv generado se guardará en la carpeta como *Servidores_C2.csv*

```
C2.txt: Bloc de notas
Archivo  Edición  Formato  Ver  Ayuda
# C2-JARM
A list of JARM hashes for different ssl implementations used by some C2 tools. Also adding other useful red team tools that use ssl (ex: EvilGinx2).

For more info on JARM hashing, check out the work by the Salesforce security team on their JARM github link here: https://github.com/salesforce/jarm

This is a neat way to fingerprint ssl servers by the software implementation. This alone would not be sufficient to detect C2 in a high fidelity manner, but JARM hashes coupled with other h

I plan to add more to this list over time. Feel free to contribute!!


| C2/RED TEAM TOOL |      SSL IMPLEMENTATION TESTED      |                              JARM HASH                              |                    LINK TO TOOL                    |
|---------|----------------------------|-----------------------------------------------------------|-----------------------------------------------|
| Mythic  | python 3 w/aiohttp 3 | 2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb | https://github.com/its-a-feature/Mythic          |
| Metasploit ssl listener | ruby 2.7.0p0 | 07d14d16d21d21d00042d43d000000aa99ce74e2c6d013c745aa52b5cc042d | https://github.com/rapid7/metasploit-framework |
| Metasploit ssl listener | ruby | 07d14d16d21d21d07c42d43d000000f50d155305214cf247147c43c0f1a823 | https://github.com/rapid7/metasploit-framework |
| Cobalt Strike | Java 11 | 07d14d16d21d21d07c42d41d00041d24a458a375eef0c576d23a7bab9a9fb1 | https://www.cobaltstrike.com/ |
| Merlin | go 1.15.2 linux/amd64 | 29d21b20d29d29d21c41d21b21b41d494e0df9532e75299f15ba73156cee38 | https://github.com/Ne0nd0g/merlin |
| Deimos | go 1.15.2 linux/amd64 with github.com/gorilla/websocket package | 00000000000000000041d00000041d9535d5979f591ae8e547c5e5743e5b64 | https://github.com/DeimosC2/DeimosC2 |
| MacC2 | python 3.8.6 w/aiohttp 3 | 2ad2ad0002ad2ad22c42d42d000000faabb8fd156aa8b4d8a37853e1063261 | https://github.com/cedowens/MacC2 |
| MacC2 | python 3.8.2 w/aiohttp 3 | 2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb | https://github.com/cedowens/MacC2 |
| MacShellSwift | python 3.8.6 socket | 2ad00000000000000000000000000000eeebf944d0b023a00f510f06a29b4f46 | https://github.com/cedowens/MacShellSwift |
| MacShell | python 3.8.6 socket | 2ad00000000000000000000000000000eeebf944d0b023a00f510f06a29b4f46 | https://github.com/cedowens/MacShellSwift |
| Sliver | go 1.15.2 linux/amd64 | 2ad2ad0002ad2ad00041d2ad2ad41da5207249a18099be84ef3c8811adc883 | https://github.com/BishopFox/sliver |
| EvilGinx2 | go 1.10.4 linux/amd64 | 20d14d20d21d20d20c20d14d20d20daddf8a68a1444c74b6dbe09910a511e6 | https://github.com/kgretzky/evilginx2 |
| Shad0w | python 3.8 flask | 2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb | https://github.com/bats3c/shad0w |
| Get2 | N/A | 07d19d12d21d21d07c07d19d07d21da5a8ab90bcc6bf8bbc6fbec4bcaa8219 | |
```

Figura 23: El archivo txt a descargar de github como *C2.txt* debe verse similar a la imágen

```
Servidores_C2.csv: Bloc de notas
Archivo  Edición  Formato  Ver  Ayuda
C2/Red Team Tool,SSL Implementation Tested,Jarm C2,Link,Fecha C2
Mythic,python3w/aiohttp3,2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb,https://github.com/its-a-feature/Mythic,2021-01-25 18:24:07.521066
Metasploitsssllistener,ruby2.7.0p0,07d14d16d21d21d00042d43d000000aa99ce74e2c6d013c745aa52b5cc042d,https://github.com/rapid7/metasploit-framework,2021-01-25 18:24:07.521066
Metasploitsssllistener,ruby,07d14d16d21d21d07c42d43d000000f50d155305214cf247147c43c0f1a823,https://github.com/rapid7/metasploit-framework,2021-01-25 18:24:07.521066
CobaltStrike,Java11,07d14d16d21d21d07c42d41d00041d24a458a375eef0c576d23a7bab9a9fb1,https://www.cobaltstrike.com/,2021-01-25 18:24:07.521066
Merlin,go1.15.2linux/amd64,29d21b20d29d29d21c41d21b21b41d494e0df9532e75299f15ba73156cee38,https://github.com/Ne0nd0g/merlin,2021-01-25 18:24:07.521066
Deimos,go1.15.2linux/amd64withgithub.com/gorilla/websocketpackage,00000000000000000041d00000041d9535d5979f591ae8e547c5e5743e5b64,https://github.com/DeimosC2/DeimosC2,2021-01-25 18:24:07.521
MacC2,python3.8.6w/aiohttp3,2ad2ad0002ad2ad22c42d42d000000faabb8fd156aa8b4d8a37853e1063261,https://github.com/cedowens/MacC2,2021-01-25 18:24:07.521066
MacC2,python3.8.2w/aiohttp3,2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb,https://github.com/cedowens/MacC2,2021-01-25 18:24:07.521066
MacShellSwift,python3.8.6socket,2ad00000000000000000000000000000eeebf944d0b023a00f510f06a29b4f46,https://github.com/cedowens/MacShellSwift,2021-01-25 18:24:07.521066
MacShell,python3.8.6socket,2ad00000000000000000000000000000eeebf944d0b023a00f510f06a29b4f46,https://github.com/cedowens/MacShellSwift,2021-01-25 18:24:07.521066
Sliver,go1.15.2linux/amd64,2ad2ad0002ad2ad00041d2ad2ad41da5207249a18099be84ef3c8811adc883,https://github.com/BishopFox/sliver,2021-01-25 18:24:07.521066
EvilGinx2,go1.10.4linux/amd64,20d14d20d21d20d20c20d14d20d20daddf8a68a1444c74b6dbe09910a511e6,https://github.com/kgretzky/evilginx2,2021-01-25 18:24:07.521066
Shad0w,python3.8flask,2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb,https://github.com/bats3c/shad0w,2021-01-25 18:24:07.521066
Get2,N/A,07d19d12d21d21d07c07d19d07d21da5a8ab90bcc6bf8bbc6fbec4bcaa8219,,2021-01-25 18:24:07.521066
```

Figura 24: El archivo csv que se ha generado con **raw_txt_to_csv.py**, llamado *Servidores_C2.csv*, debe verse similar a esto

Es este archivo csv el que se utilizará para hacer el inner join con los dominios .CL extraidos de NIC Chile, aportando los fingerprint de Jarm maliciosos para poder hacer el match.

Por otro lado, si se tiene guardado como *domains.csv* el archivo que contiene todos los dominios .CL de NIC Chile, para tener los fingerprints de estos primero hay que clonar el repositorio de Jarm (enlace 1 del anexo) y luego utilizar el comando de consola: **$bash jarm.sh domains.csv resultado_jarm.csv**.

Esto hará un fingerprint de Jarm para cada dominio del csv *domains.csv* y generará el archivo csv llamado *resultado_jarm.csv*. Ojo que todos estos archivos deben estar en la misma carpeta, o bien no funcionarán adecuadamente los comandos, en caso contrario se debe especificar también la ubicación en el nombre del archivo.

Una vez obtenidos tanto el archivo csv de los servidores C2 llamado **Servidores_C2.csv** y el de los dominios .CL llamado **resultado_jarm.csv** podemos utilizar el archivo de python llamado **jarm_chile.py** para generar el csv con los dominios .CL potencialmente C2. El resultado de este inner join de dominios .CL y servidores C2 será un archivo csv llamado **Botnets_red_chilena.csv**

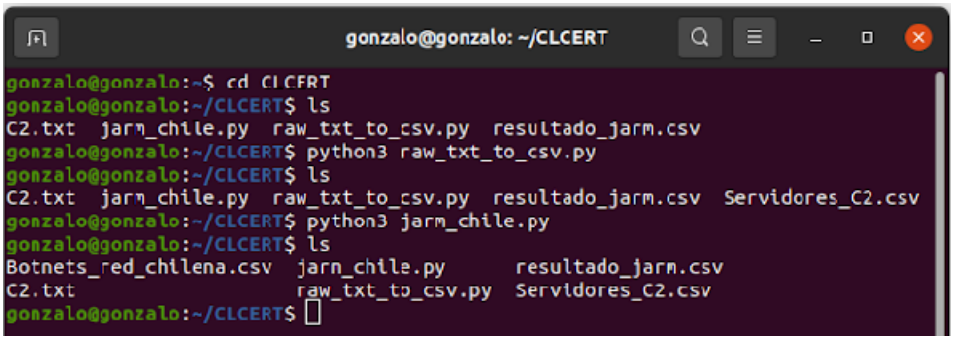

Figura 25: Ejemplo de uso

En el ejemplo de uso anterior solo falta la parte de generar el csv *resultados_jarm.csv* mediante jarm.py, pero eso es sólo seguir esos pasos mencionados anteriormente. También es importante señalar que todos los archivos deben estar en la misma carpeta para poder funcionar, en este caso la carpeta se llama CLCERT. Los códigos tanto de *raw_txt_to_csv.py* como *jarm_chile.py* se adjuntan en el anexo.

Básicamente, y a modo de resumen conceptual, lo que se hace es buscar los siguientes fingerprints de Jarm de servidores C2 en la lista de dominios .CL y ver cuales coinciden.

- trickbot = 22b22b09b22b22b22b22b22b22b22b22b352842cd5d6b0278445702035e06875c
- Metasploit = 07d14d16d21d21d00042d43d000000aa99ce74e2c6d013c745aa52b5cc042d
- Metasploit2 = 07d14d16d21d21d07c42d43d000000f50d155305214cf247147c43c0f1a823
- CobaltStrike = 07d14d16d21d21d07c42d41d00041d24a458a375eef0c576d23a7bab9a9fb1
- Merlin = 29d21b20d29d29d21c41d21b21b41d494e0df9532e75299f15ba73156cee38
- Mythic = 2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb
- Deimos = 00000000000000000041d00000041d9535d5979f591ae8e547c5e5743e5b64
- MacC2 = 2ad2ad0002ad2ad22c42d42d000000faabb8fd156aa8b4d8a37853e1063261
- MacC2_2 = 2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb
- MacShellSwift = 2ad00000000000000000000000000eeebf944d0b023a00f510f06a29b4f46
- MacShell = 2ad00000000000000000000000000eeebf944d0b023a00f510f06a29b4f46
- Sliver = 2ad2ad0002ad2ad00041d2ad2ad41da5207249a18099be84ef3c8811adc883
- EvilGinx2 = 20d14d20d21d20d20c20d14d20d20daddf8a68a1444c74b6dbe09910a511e6
- Shad0w = 2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e29eecb81d0c7b06eb
- Get2 = 07d19d12d21d21d07c07d19d07d21da5a8ab90bcc6bf8bbc6fbec4bcaa8219

5. Resultados

A nivel exploratorio de los dominios .CL tenemos que son 626.681 dominios, de los cuales hay 292.439 activos según el resultado de utilizar Jarm.py

En estos 292.439 dominios existen solamente 2.060 fingerprints distintos (hay varios repetidos).

```
cantidad_fingerprints = len(df_dominios['Jarm'].unique().tolist())
cantidad_fingerprints
```

```
2060
```

Figura 26: Cantidad de fingerprints distintos en los dominios .CL

Y obviamente hay algunos repetidos más veces que otros:

```
df_dominios["Jarm"].value_counts()
```

```
29d29d15d29d29d00042d42d000000cd600c085f371f8533aaf66051f8e5b1    31039
25d3fd00025d25d00042d43d0000001b0c486b0c0bae33c7ce67177fe8bab5    23885
15d3fd16d29d29d00042d43d0000001ae0802418786940cae38f1d9eed5b9b    19765
29d29d20d29d29d00041d29d29d41dc78d825a1d7337f1083c6606d7fa07c6    17746
29d29d00029d29d00042d43d00041dd469afa8cfbe5e42c631eb3fc55d6787    16923
                                                                  ...
28d28d28d0000001ec28d28d28d28dbc07bc352a188303b518c8a273c71220        1
25d25d00025d25d06c25d25d25d25d3935235a12456c2911a50cf12a624491        1
26d26d00026d26d22c42d42d0000005b002cb46912a7d649e6bd117d8b41c5        1
29d29d00000000022c29d00000029dfe50d79957149ddcd0cc019ff0a0b6a9        1
05b02b20b21b20b05b05b02b05b20b74fcf6501ae7a92319e575bfafd2a827        1
Name: Jarm, Length: 2060, dtype: int64
```

Figura 27: Frecuencia de algunos fingerprints en los dominios .CL

Lo que significa que hay algunos fingerprints, como el primero de la lista, que aparecen hasta 31.039 veces en los dominios .CL, mientras que otros como el último aparecen solo una vez.

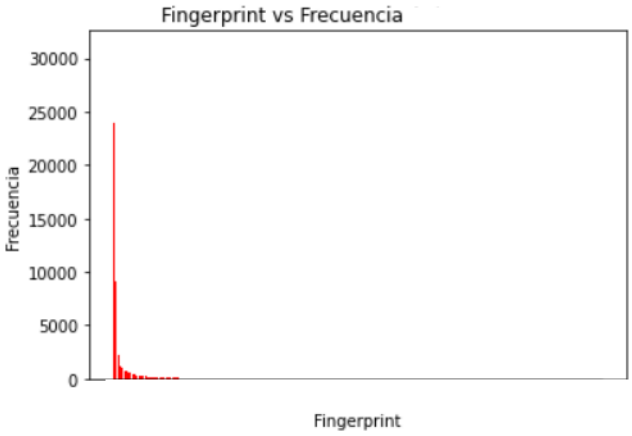

Figura 28: Idea gráfica de que hay algunos fingerprint muy comunes (outliers)

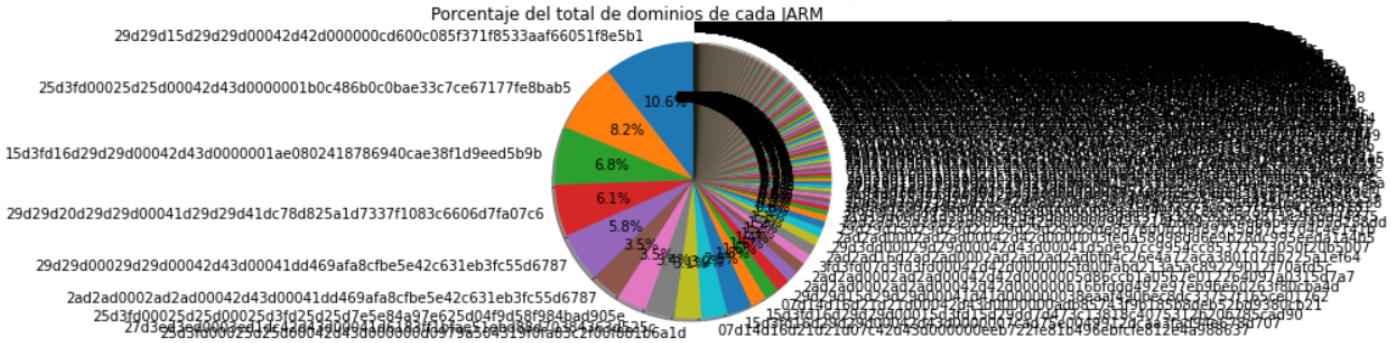

Figura 29: Idea gráfica de que hay algunos fingerprint muy comunes (outliers)

Hay mucho trabajo que se puede hacer a la hora de explorar todos los dominios .CL como ver el comportamiento de los fingerprints de Jarm completos, o los primeros 30 caracteres, o los segundos 32, o bien cada una de las respuestas a los saludos TLS. De hecho una de las cosas que se propone hacer es un análisis a fondo de las distribuciones de los resultados de los 10 saludos TLS que hace Jarm en la red de dominios .CL.

Por ejemplo se pueden ver cuáles son los saludos TLS más comunes en la red chilena para el primer saludo de Jarm, para el segundo y así, o bien ver los más raros / poco frecuentes, ¿en qué dominios ocurren los resultados más raros de TLS?, ¿tienen algo en común esos dominios?, ¿el código fuente de Jarm disponible en github nos dice algo acerca de qué significan esos saludos TLS más raros en particular?. Por simplicidad sólo veremos qué tan raros son los saludos TLS de los dominios .CL que tengan un fingerprint malicioso, resultado que se mostrará un poco más adelante.

Lo primero que queremos realizar es un inner join entre los fingerprints de los dominios .CL y los fingerprints de servidores C2 conocidos hasta el momento.

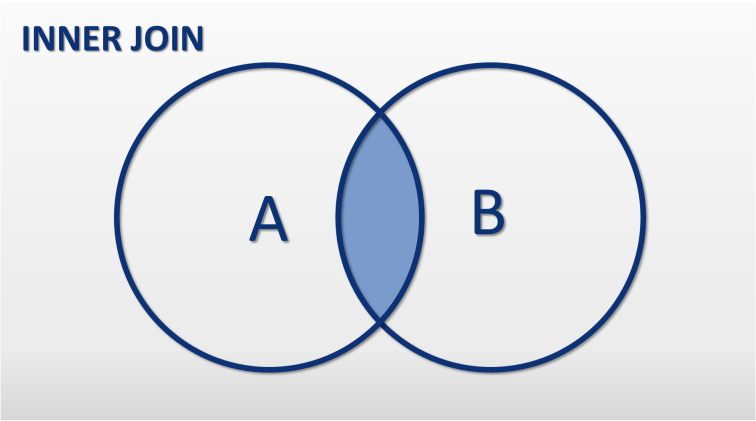

Figura 30: Conjunto A: Total dominios .CL con sus fingerprints, Conjunto B: fingerprints de servidores C2

Si nuestro conjunto A es el total de dominios activos .CL (extraído de NIC Chile)

	Dominio	IP	Jarm
1	007gps.cl	190.4.193.170	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...
4	0101.cl	190.107.177.235	25d3fd00025d25d00025d3fd25d25d7e5e84a97e625d04...
5	0000.cl	168.195.128.167	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...
7	009.cl	200.35.157.92	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...
14	06publicidad.cl	201.148.107.13	2ad2ad0002ad2ad22c2ad2ad2ad2ad6a7bd8f51d54bfc0...
...	...	...	...
626588	zuletaygarciacontadores.cl	131.72.236.33	07d14d16d21d21d07c42d43d000000eeb722fe81b496eb...
626597	zuntao.cl	131.72.236.178	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...
626604	zuritapublicidad.cl	131.72.236.63	15d3fd16d29d29d00042d43d0000001ae0802418786940...
626606	zuvar.cl	131.72.236.168	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...
626658	zonanotebook.cl	131.72.236.58	25d3fd00025d25d00042d43d0000001b0c486b0c0bae33...

292439 rows × 3 columns

Figura 31: Conjunto A: Total dominios .CL

Y nuestro conjunto B es la lista actualizada de fingerprints de servidores C2 (extraído del github proporcionado por Jhon Althouse)

	C2/Red Team Tool	SSL Implementation Tested	Jarm C2
0	Mythic	python3w/aiohttp3	2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e2...
1	Metasploitssllistener	ruby2.7.0p0	07d14d16d21d21d00042d43d000000aa99ce74e2c6d013...
2	Metasploitssllistener	ruby	07d14d16d21d21d07c42d43d000000f50d155305214cf2...
3	CobaltStrike	Java11	07d14d16d21d21d07c42d41d00041d24a458a375eef0c5...
4	Merlin	go1.15.2linux/amd64	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...
5	Deimos	go1.15.2linux/amd64withgithub.com/gorilla/webs...	00000000000000000041d00000041d9535d5979f591ae8...
6	MacC2	python3.8.6w/aiohttp3	2ad2ad0002ad2ad22c42d42d000000faabb8fd156aa8b4...
7	MacC2	python3.8.2w/aiohttp3	2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e2...
8	MacShellSwift	python3.8.6socket	2ad00000000000000000000000000eeebf944d0b023a0...
9	MacShell	python3.8.6socket	2ad00000000000000000000000000eeebf944d0b023a0...
10	Sliver	go1.15.2linux/amd64	2ad2ad0002ad2ad00041d2ad2ad41da5207249a18099be...
11	EvilGinx2	go1.10.4linux/amd64	20d14d20d21d20d20c20d14d20d20daddf8a68a1444c74...
12	Shad0w	python3.8flask	2ad2ad0002ad2ad00042d42d000000ad9bf51cc3f5a1e2...
13	Get2	NaN	07d19d12d21d21d07c07d19d07d21da5a8ab90bcc6bf8b...

Figura 32: Conjunto B: fingerprints de servidores C2

Nuestro resultado del inner join será el siguiente conjunto de dominios .CL

Dominio	IP	Jarm	C2/Red Team Tool	SSL Implementation Tested	Jarm C2
growto.cl	198.185.159.144	00000000000000000041d00000041d9535d5979f591ae8...	Deimos	go1.15.2linux/amd64withgithub.com /gorilla/webs...	00000000000000000041d00000041d9535d5979f591ae8...
iglesiacasadelpadre.cl	143.110.147.106	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...	Merlin	go1.15.2linux/amd64	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...
jovenescp.cl	143.110.147.106	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...	Merlin	go1.15.2linux/amd64	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...
kingdom.cl	143.110.147.106	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...	Merlin	go1.15.2linux/amd64	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...
lasbrujasdetalagante.cl	95.179.243.224	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...	Merlin	go1.15.2linux/amd64	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...
prettify.cl	192.241.132.157	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...	Merlin	go1.15.2linux/amd64	29d21b20d29d29d21c41d21b21b41d494e0df9532e7529...

Figura 33: Dominios .CL que su fingerprint coincide con el de un servidor C2

Finalmente si vemos la frecuencia de los saludos TLS de Jarm para estos dominios tenemos lo siguiente:

Dominio	IP	TLS 1 %	TLS 2 %	TLS 3 %	TLS 4 %	TLS 5 %	TLS 6 %	TLS 7 %	TLS 8 %	TLS 9 %	TLS 10 %	primeros 30 %	segundos 32 %
growto.cl	198.185.159.144	0.118315	0.104979	39.674941	4.793136	0.495146	75.240648	9.028208	0.062235	68.144810	7.381368	0.000342	0.006497
iglesiacasadelpadre.cl	143.110.147.106	0.186022	0.008891	0.441802	0.187048	0.187048	0.008891	9.028208	0.008891	0.008891	7.381368	0.002736	0.432569
jovenescp.cl	143.110.147.106	0.186022	0.008891	0.441802	0.187048	0.187048	0.008891	9.028208	0.008891	0.008891	7.381368	0.002736	0.432569
kingdom.cl	143.110.147.106	0.186022	0.008891	0.441802	0.187048	0.187048	0.008891	9.028208	0.008891	0.008891	7.381368	0.002736	0.432569
lasbrujasdetalagante.cl	95.179.243.224	0.186022	0.008891	0.441802	0.187048	0.187048	0.008891	9.028208	0.008891	0.008891	7.381368	0.002736	0.432569
prettify.cl	192.241.132.157	0.186022	0.008891	0.441802	0.187048	0.187048	0.008891	9.028208	0.008891	0.008891	7.381368	0.002736	0.432569

Figura 34: Dominios .CL potencialmente C2 y qué tan raros son sus saludos TLS, sus primeros 30 caracteres y sus segundos 32 caracteres

¿Qué significan esos porcentajes? básicamente lo que se hizo fue dividir el total de apariciones del saludo TLS en concreto sobre el total de saludos TLS de los dominios .CL y multiplicar por 100 para llevar a porcentaje. Es así como cada porcentaje muestra cuanta parte de ”la torta” del total de saludos TLS representa ese saludo en concreto (similar a lo mostrado en la figura 29 con los fingerprint de Jarm).

Vemos así que en general son saludos TLS muy poco comunes en la red de dominios .CL, al igual que sus primeros 30 caracteres y sus segundos 32 caracteres. Por un lado tiene sentido ya que son fingerprints muy poco comunes en la red los que coincidieron con servidores C2 ya que son sólo 6 dominios, pero puede llegar a ser interesante comenzar a ahondar en estas distribuciones y ver si tienen mayores implicancias o bien sólo es un añadido sin más.

En el entregable de github también se adjunta un jupyter notebook de ejemplo para la exploración de datos con pandas.

6. Análisis

Una buena idea luego de obtener el resultado del inner join entre los dominios .CL y los servidores C2, es ver si tienen algo en común esos dominios sospechosos. En nuestro caso simplemente haremos una pequeña investigación acerca de sus ASN, pero queda propuesto para futuros estudios ver más formas de análisis, quizás de puertos con Nmap u otros softwares.

Para el análisis utilizaremos la página ipinfo.io. Vemos que 3 dominios (50 % de los dominios sospechosos) corresponden a la misma IP 143.110.147.106, por lo que partimos el análisis por ellos (iglesiacasadelpadre.cl, jovenescp.cl y kingdom.cl).

Esta dirección IP pertenece al ASN Digital Ocean, LLC:

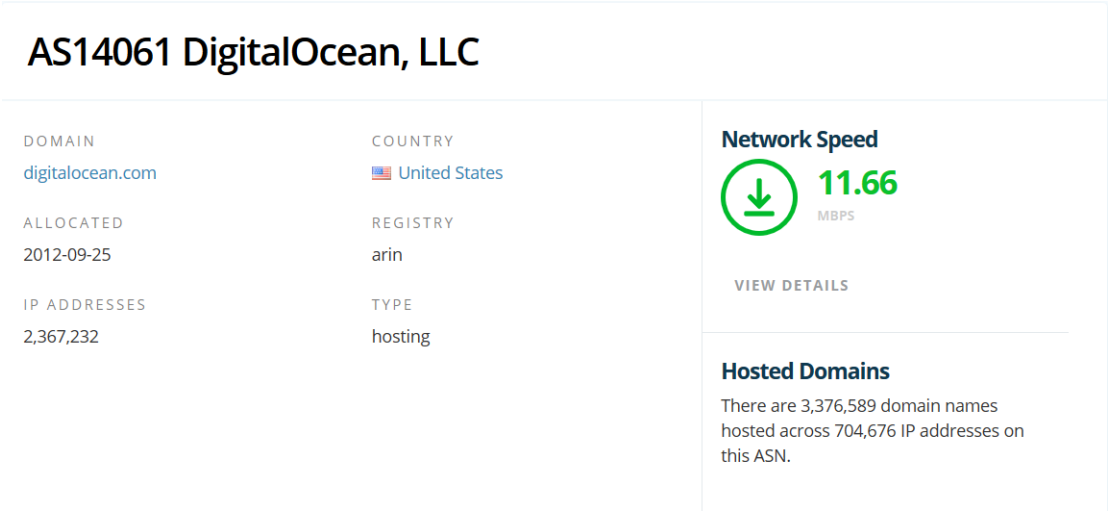

Figura 35: ASN asociado a la IP 143.110.147.106

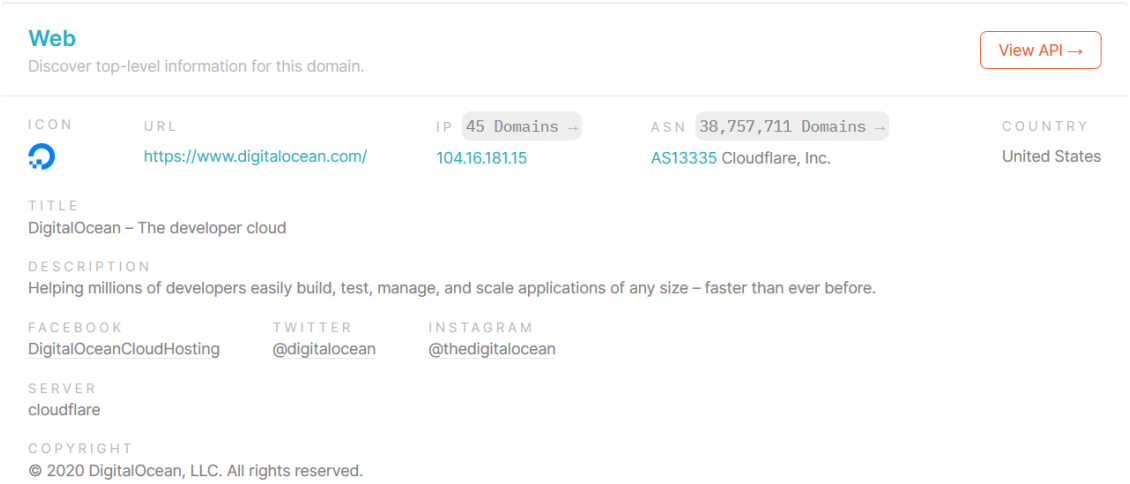

Figura 36: Más información acerca del ASN asociado a la IP 143.110.147.106

Si vemos la cantidad de dominios asociados a la IP 143.110.147.106, vemos que son sólo los 3 que encontramos en la red de dominios .CL (jovenescp.cl, iglesiacasadelpadre.cl, kingdom.cl).

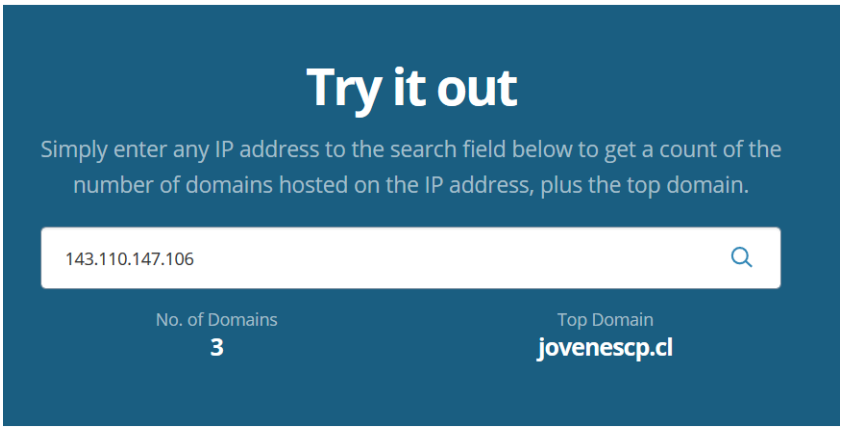

Figura 37: Cantidad de Dominios asociados a la IP 143.110.147.106

Si continuamos y vemos el ASN asociado a la IP 192.241.132.157 del dominio prettify.cl vemos que es el mismo ASN de antes: Digital Ocean (por lo que no agregaremos nuevamente la información del ASN en cuestión).

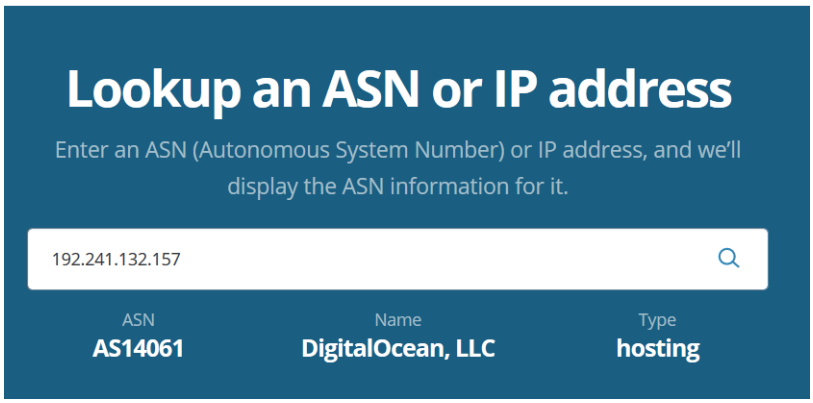

Figura 38: ASN asociado a la IP 192.241.132.157

Mientras que la cantidad de dominios asociados a esta IP 192.241.132.157 hospedada en Digital Ocean es de solamente 2 dominios:

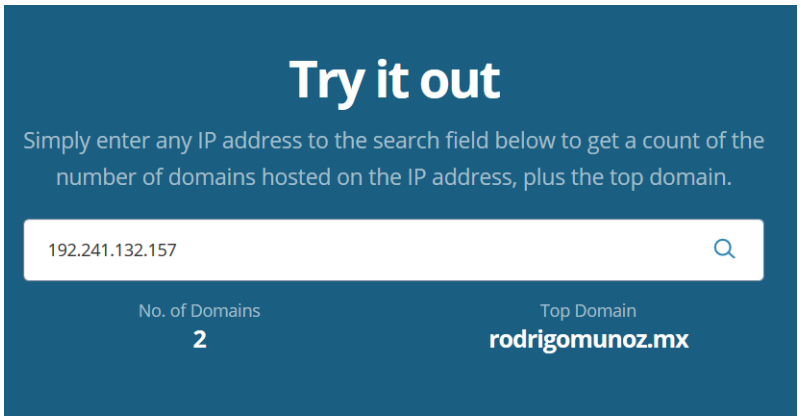

Figura 39: Cantidad de Dominios asociado a 192.241.132.15

Esto se hace un poco raro ya que en 2 direcciones IPs distintas Digital Ocean incurre en lo mismo, tiene pocos dominios y casi todos con fingerprint de Jarm sospechosos. Decimos casi ya que faltaría ver el otro dominio asociado a la IP 192.241.132.157, el sitio rodrigomunoz.mx, puede que también posea un Jarm igual al de prettify.cl y esto ya sería bastante extraño.

Pasando ahora a la IP 198.185.159.144 del dominio growto.cl, vemos que el ASN asociado es Squarespace, Inc.

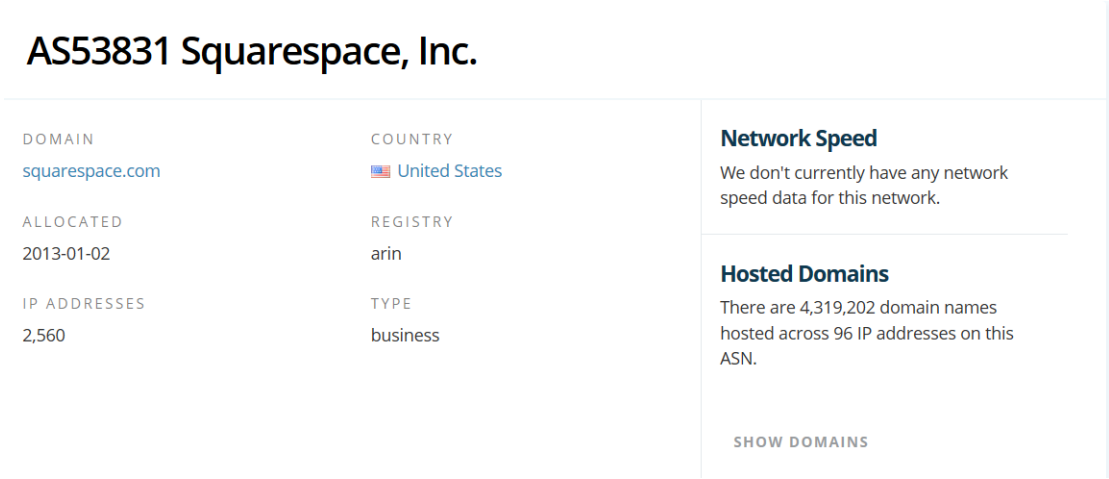

Figura 40: ASN asociado a la IP 198.185.159.144

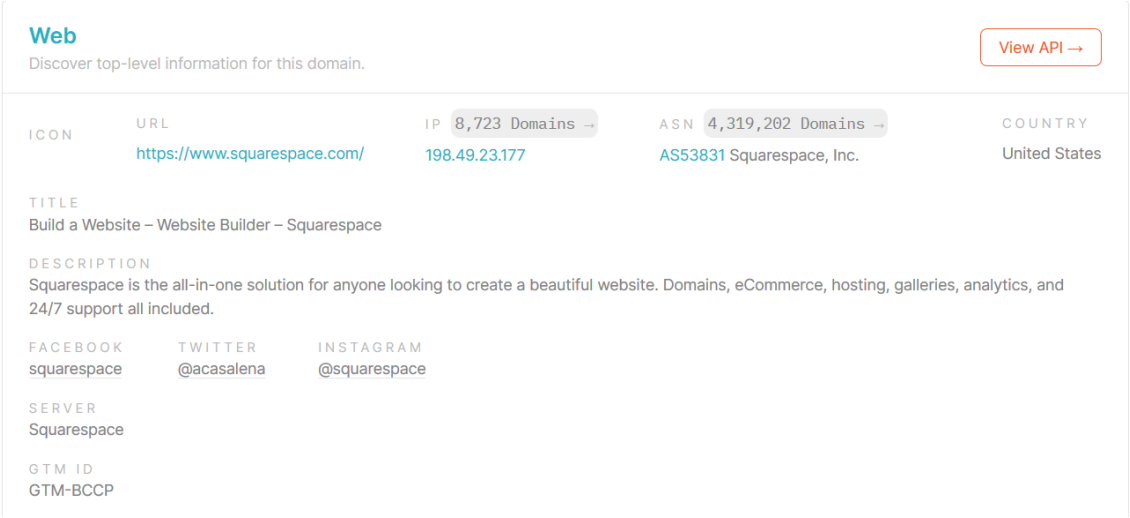

Figura 41: Más información acerca del ASN asociado a la IP 198.185.159.144

Si vemos la cantidad de dominios asociados a la IP 143.110.147.106, vemos que son ahora 3.671.545 los que encontramos, cantidad mucho más elevada que los casos relacionados a Digital Ocean.

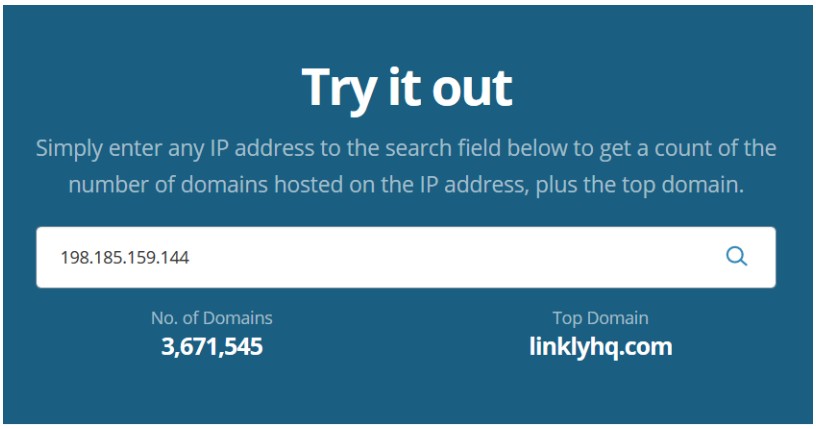

Figura 42: Cantidad de Dominios asociados a la IP 198.185.159.144

Finalmente vemos el ASN asociado a la IP 95.179.243.224 del dominio lasbrujasdetalagante.cl, el cual es Choopa, LLC.

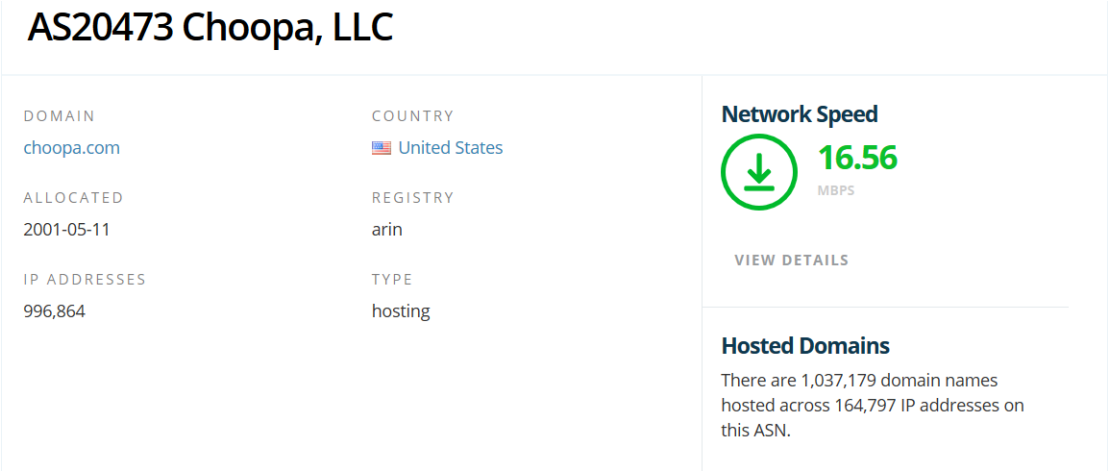

Figura 43: ASN asociado a la IP 95.179.243.224

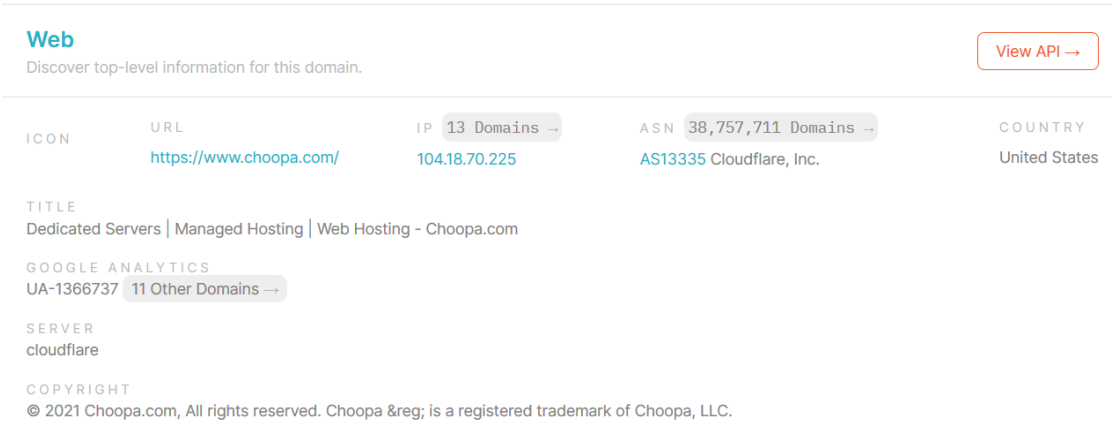

Figura 44: Más información acerca del ASN asociado a la IP 95.179.243.224

Si vemos la cantidad de dominios asociados a la IP 95.179.243.224, vemos que son ahora 4.537 los que encontramos, un termino medio entre los 2 casos de ASN anteriores.

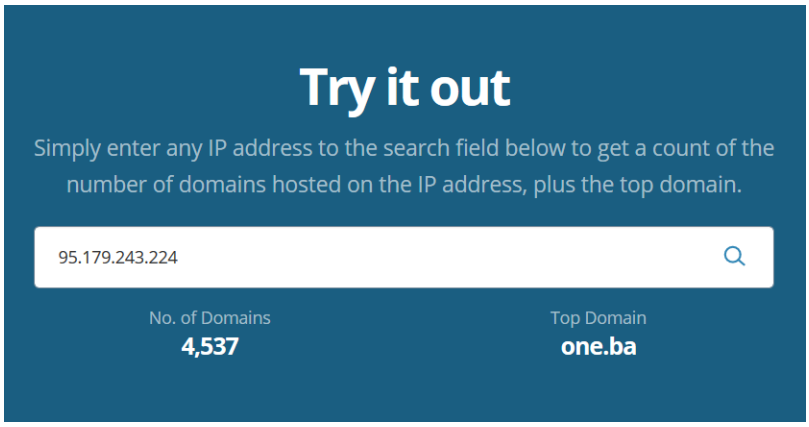

Figura 45: Cantidad de Dominios asociados a la IP 95.179.243.224

Lamentablemente es poco lo que se puede concluir con este análisis, pero se extiende la invitación a otros mayores y más exhaustivos. Por ejemplo puede ser ver los puertos abiertos de estos servidores, ver a fondo sus respuestas TLS, quién les proporciona sus certificados TLS, sus registros en NIC Chile, entre otras cosas. Un fuerte análisis posterior de los dominios .CL puede ayudar a eliminar falsos positivos de la lista de sospecha, ya que recordemos que Jarm por si sólo no es suficientemente concluyente para asegurar que un dominio sea un servidor C2.

7. Alcances y Conclusiones

El uso de Jarm en la búsqueda de servidores C2 no sólo representa una nueva herramienta para identificar potenciales servidores C2, si no que también es una potencial herramienta para entender de mejor forma los protocolos TLS que estos servidores utilizan. El fijarse en los protocolos TLS para generar el fingerprint es algo astuto a día de hoy, ya que es innovador y no muchos hackers los deben estar modificando a día de hoy en sus servidores C2, pero por otro lado es una carrera contra el tiempo ya que ante cada herramienta surgen medidas para burlarla.

Por otro lado el funcionamiento de Jarm es interesante, ya que proporciona métricas que antes no habían. Estas métricas surgen de su funcionamiento, el cómo mediante bloques (saludos TLS) se conforman los primeros 30 y segundos 32 caracteres del fingerprint. Actualmente el enfoque es de 1 a 1 con fingerprints maliciosos (es decir 62 caracteres iguales entre el fingerprint del dominio .CL y el de la lista de servidores C2) pero esto podría mutar a ver quizás componentes escenciales dentro de esos 10 saludos TLS, quizás hay algunos más representativos que otros, quizás hay formas de ver esto que no sean 1 a 1 si no que con porcentajes de similitud entre los 10 saludos TLS. Quizás hay partes del protocolo TLS que no pueden ser mutadas por los servidores C2 ni ahora ni en el futuro ya que es algo intrínseco de su funcionamiento, lo que haría que quizás hayan bloques de los saludos del Jarm que si o si van a estar presentes en un servidor C2.

Dada la duración de la práctica no se pudo ahondar más en la distribución de las frecuencias de los distintos saludos TLS de Jarm, tampoco en qué significa cada uno, pero las posibilidades de la herramienta son muchas. Quizás aún está muy verde ya que acaba de salir hace un par de meses, pero la verdad es que es cosa de adentrarse en ella, aprender bien como funciona a partir de su código fuente y ver lo que esto permite como herramienta.

Una de las limitaciones a la hora de utilizar Jarm actualmente es que su mayor utilidad es utilizando dominios, como bien vimos en el primer enfoque de solución. El problema es que es muy difícil encontrar dominios de servidores C2 como ya vimos en el segundo enfoque de solución (ver documento de IBM, enlace 10 del anexo). Es por esto que se hace complejo tener una lista de fingerprints de servidores C2 como la mostrada en el github propocionado por John Althouse, para así hacer el inner join con los dominios .CL (tercer enfoque de solución).

Se propone buscar nuevas formas de encontrar esos fingerprints maliciosos en vez del github proporcionado, también estudiar e intentar buscar las partes de los saludos TLS de Jarm claves para identificar a un servidor maligno (con el código fuente de Jarm). Por otro lado, hay harto trabajo en la exploración de los saludos TLS en la red de dominios chilena, ver cómo se distribuyen, los más comunes, los más raros, que tienen en común los sospechosos, etc. Finalmente se propone también mejorar la exploración de los dominios entregados en el archivo **Botnets_red_chilena.csv**, el análisis expuesto acá es pequeño y no concluyente, pero se puede extender utilizando más y mejores herramientas.

8. Anexos

8.1. Correos con John Althouse

Con John Althouse (creador de Jarm) se intercambiaron un par de correos con dudas, las respuestas se adjuntan a continuación. Recomiendo mucho en caso de haber preguntas hacérselas a él a su correo, me dió la impresión de tener muy buena disposición a la hora de hablar de Jarm y las mejores maneras de utilizarlo.

John Althouse <jalthouse@salesforce.com> lun, 4 ene 18:09
para mí

inglés > español Traducir mensaje Desactivar para: inglés

Hey, good questions!

Yeah, we need to update the JARM code to follow redirects, hopefully we can get to that this month.

I recommend scanning IPs and domains where applicable because they can produce different results. A single host can have multiple TLS servers running on it and the host can decide which application to route the traffic to depending on the domain name in the SNI field of the TLS client hello. So in the instance a host is setup like this, if you scan the IP, you'll get the JARM of the host. If you scan the domain, you may get a different JARM of the application for that domain, depending. All of this is good information to have.

JOHN ALTHOUSE, OSCE

Figura 46: Primer correo de Jhon Althouse

John Althouse mar, 19 ene 19:55
para mí

inglés > español Traducir mensaje Desactivar para: inglés

Hey Gonzalo,

As I said in the blog post, JARM should not be used as a sole means of identifying malicious hosts, you should use it in conjunction with other metadata to build a likelihood of maliciousness. For example, the JARM in the blog post for Cobalt Strike, it turns out to be the JARM for the particular version of Java that Cobalt Strike uses. So any servers running that version of Java would be identified with JARM, then you would have to run extra checks to see if it's Cobalt Strike or not. In this blog, https://www.randhome.io/blog/2020/12/20/analyzing-cobalt-strike-for-fun-and-profit/, the researcher used a tool that dumps Cobalt Strike configs to confirm that the hosts are indeed Cobalt Strike after scanning for the JARM.

Here's a list of JARM to C2s and their SSL implementation: https://github.com/cedowens/C2-JARM

JOHN ALTHOUSE, OSCE

Figura 47: Segundo correo de Jhon Althouse

8.2. Enlaces de interés

[1] Github Jarm
https://github.com/salesforce/jarm

[2] Malware en conexiones TLS
https://www.imperial.ac.uk/media/imperial-college/faculty-of-engineering/computing/public/1819-pg-projects/Detecting-Malware-in-TLS-Traf%EF%AC%81c.pdf

[3] Tesis: Nuevas estrategias de análisis de datos de escaneos de la red chilena para el monitoreo periódico de su seguridad, Eduardo Riveros
http://repositorio.uchile.cl/handle/2250/175511

[4] Publicación de Jarm, Salesforce Engineering
https://engineering.salesforce.com/easily-identify-malicious-servers-on-the-internet-with-jarm-e095edac525a

[5] maxmind https://www.maxmind.com/en/home

[6] Zmap https://github.com/zmap/zmap

[7] Feodo Tracker https://feodotracker.abuse.ch/

[8] Zones Files. Lista de dominios comprometidos https://zonefiles.io/compromised-domain-list/

[9] Github de cedowens que tiene una lista de fingerprints C2. https://github.com/cedowens/C2-JARM

[10] IBM The inside story on botnets https://www.ibm.com/downloads/cas/V3YJVYZX

[11] IPInfo.io https://ipinfo.io/

8.3. Códigos

Código 1: raw_txt_to_csv.py

```
import datetime
from pandas import DataFrame

#Abrimos el archivo txt con los servidores C2 y sus Jarm
a_file = open("C2.txt", "r")

#Creamos una lista vacia
list_of_lists = []
#Guardamos cada linea del archivo txt como una lista en la lista list_of_lists (lista de listas)
for line in a_file:
  stripped_line = line.strip()
  line_list = stripped_line.split()
  list_of_lists.append(line_list)

a_file.close()

#Encontramos la primera aparicion del string |, para llegar a la tabla donde estan los datos útiles
#posicion guarda la fila del archivo C2.txt en la que se comienza a leer la tabla
i=0
while i<=len(list_of_lists):
  if list_of_lists[i]!=[]:
    if list_of_lists[i][0]=="|":
      posicion=i
      i=27
    else:
      i=i+1
  else:
    i=i+1

#texto equivale a list_of_lists anterior, pero solo guardando en cada lista los valores de cada fila de la tabla (ya no hay nada del texto original de introduccion)
texto=[]
for i in range(posicion,len(list_of_lists)):
  texto.append(list_of_lists[i])

#convertimos a strings de texto cada lista, así solo tenemos una lista de strings
csv=[]
for i in texto:
  listToStr = ' '.join(map(str, i))
  csv.append(listToStr)

#hacemos un split para separar los campos relevantes que en la tabla estaban delimitados por el string |
#generamos nuevamente una lista de listas, similar a la encontrada en texto, pero con los campos bien separados e identificados y sin el caracter |
csv2=[]
for i in csv:
  csv2.append(i.split('|'))
```

```

#generamos nuevamente una lista, aunque ahora eliminamos los campos vacios ""
csv3=[]
for i in csv2:
  csv3.append(i[1:len(i)-1])

#eliminamos las 2 primeras listas (o filas de la tabla), ya que una son los headers mal hechos y otra son puros caracteres separadores -----
csv3=csv3[2:len(csv3)]

#eliminamos los espacios de los strings, para que se puedan comparar bien con el fingerprint de los dominios
#p =" Hola "
#y = p.replace(" ","") -> "Hola"
for i in range(0,len(csv3)):
  for j in range(0,len(csv3[i])):
    csv3[i][j] = csv3[i][j].replace(' ','')

#Lo convertimos en un df

df = DataFrame(csv3,columns=['C2/Red Team Tool','SSL Implementation Tested','Jarm C2','Link'])
fecha = datetime.datetime.now()
df["Fecha C2"]=fecha

#Descargamos el df
df.to_csv('Servidores_C2.csv',index=False) #index se puede dejar como false para mantenerlos o quitar para que salga sin los header
```

Código 2: jarm_chile.py

```
import pandas as pd
import operator
import datetime

#Se crea el dataframe a partir del csv
df=pd.read_csv("resultado_jarm.csv")

#Se añaden los headers de las columnas al csv
df.columns=["Dominio","IP","Jarm","TLS 1","TLS 2","TLS 3","TLS 4","TLS 5","TLS 6","TLS 7","TLS 8","TLS 9","TLS 10"]

#Filtramos los dominios que tienen fingerprint 0 o bien no se pudo hacer conexion
sinconexion = str(str(0)*62)
df2 = df[df["IP"] != "Failed to resolve IP"]
df_dominios = df2[df2["Jarm"] != str(sinconexion)]

#Eliminamos las columnas de los saludos TLS que no nos importan/aportan nada
df_dominios = df_dominios.drop(["TLS 1","TLS 2","TLS 3","TLS 4","TLS 5","TLS 6","TLS 7","TLS 8","TLS 9","TLS 10"], axis=1)

#Se crea el dataframe de las botnets conocidas a partir del csv
df_C2=pd.read_csv("Servidores_C2.csv")
```

```python

#Se crea el inner join entre ambos dataframes
inner_join = pd.merge(left=df_dominios,right=df_C2, left_on='Jarm', right_on='Jarm C2')

#Se agrega el campo de fecha
fecha = datetime.datetime.now()
inner_join["Fecha"]=fecha

#Se elimina el campo repetido del Jarm
resultado_final = inner_join.drop(["Jarm C2"], axis=1)

#df.to_csv(r'Path where you want to store the exported CSV file\File Name.csv')
#df_output.to_csv(r'C:\Users\gonza\Desktop\Botnets_red_chilena.csv') seria en caso de no estar corriendo en colab
resultado_final.to_csv('Botnets_red_chilena.csv',index=False) #index se puede dejar como false para mantenerlos o quitar para que salga sin los header
```